

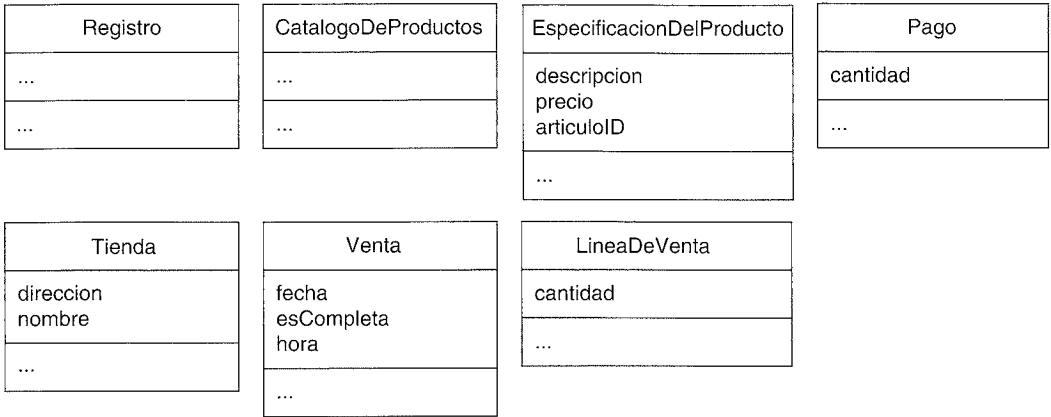


Figura 19.3. Clases software de la aplicación.

Nótese que algunos de los conceptos del Modelo del Dominio, como *Cajero*, no están presentes en el diseño. No es necesario —para la iteración actual— representarlos en el software. Sin embargo, en iteraciones posteriores, cuando se aborden nuevos requisitos y casos de uso, podrían formar parte del diseño. Por ejemplo, cuando se implementen los requisitos de seguridad y de inicio de sesión, es probable que sea relevante una clase software denominada *Cajero*.

Añadir los nombres de los métodos

Se pueden identificar los nombres de los métodos analizando los diagramas de interacción. Por ejemplo, si se envía el mensaje *crearLineaDeVenta* a una instancia de la clase *Venta*, entonces la clase *Venta* debe definir un método *crearLineaDeVenta* (ver Figura 19.4).

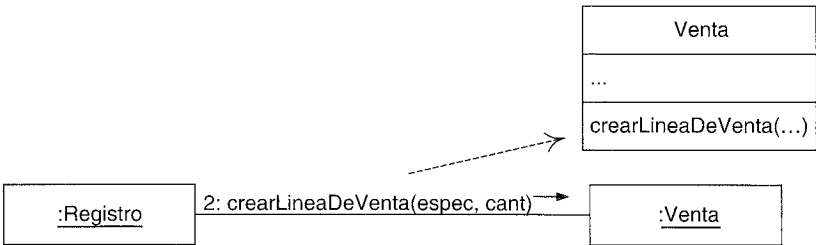


Figura 19.4. Nombres de los métodos a partir de los diagramas de interacción.

En general, el conjunto de todos los mensajes enviados a una clase X a lo largo de todos los diagramas de interacción indican la mayoría de los métodos que debe definir la clase X.

La inspección de todos los diagramas de interacción de la aplicación del PDV da lugar a la asignación de métodos que se muestra en la Figura 19.5.

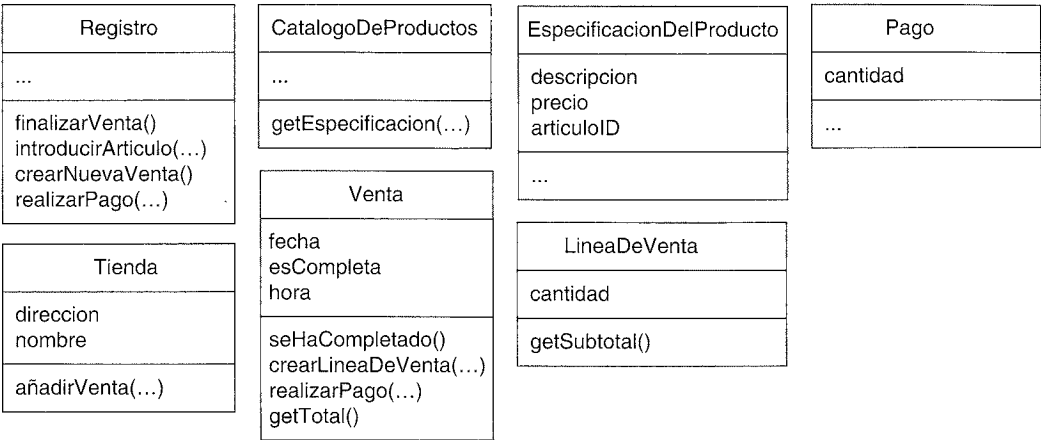


Figura 19.5. Métodos de la aplicación.

Cuestiones acerca de los nombres de los métodos

Se deben tener en cuenta las siguientes cuestiones especiales en relación con los nombres de los métodos:

- Interpretación del mensaje *create*.
- Descripción de los métodos de acceso.
- Interpretación de los mensajes a los multiobjetos.
- Sintaxis dependiente del lenguaje.

Nombres de los métodos—create

El mensaje *create* es una forma independiente del lenguaje posible en UML para indicar instanciación e inicialización. Al traducir el diseño a un lenguaje de programación orientado a objetos, se debe expresar en función de sus estilos para la instanciación e inicialización. No existe un método *create* ni en C++, ni en Java, ni en Smalltalk. Por ejemplo, en C++, implica la asignación automática, o asignación de almacenamiento libre con el operador *new*, seguido de una llamada al constructor. En Java, implica la invocación del operador *new*, seguido de una llamada al constructor.

Debido a las múltiples interpretaciones, y también debido a que la inicialización es una actividad muy común, es habitual omitir en el DCD los métodos relacionados con la creación y los constructores.

Nombres de los métodos—métodos de acceso

Los **métodos de acceso** recuperan (método de obtención, *accessor*) o establecen (método de cambio, *mutador*) el valor de los atributos. En algunos lenguajes (como Java) es un estilo común tener un *accessor* y un *mutador* para cada atributo, y declarar todos los

atributos como privados (para imponer la encapsulación de datos). Normalmente, se excluye la descripción de estos métodos en el diagrama de clases debido a que generan mucho ruido; para n atributos, hay $2n$ métodos sin interés. Por ejemplo, no se muestra, aunque está presente, el método *getPrecio* (o *precio*) de la *EspecificacionDelProducto*, porque se trata simplemente de un método de acceso.

Nombres de los métodos—multiobjetos

Un mensaje a un multiobjeto se interpreta como un mensaje al propio objeto contenedor/colección. Por ejemplo, el siguiente mensaje al multiobjeto *buscar* se debe interpretar como un mensaje al objeto contenedor/colección, como a un *Map* en Java, un *map* en C++, o un *Dictionary* en Smalltalk (ver Figura 19.6).

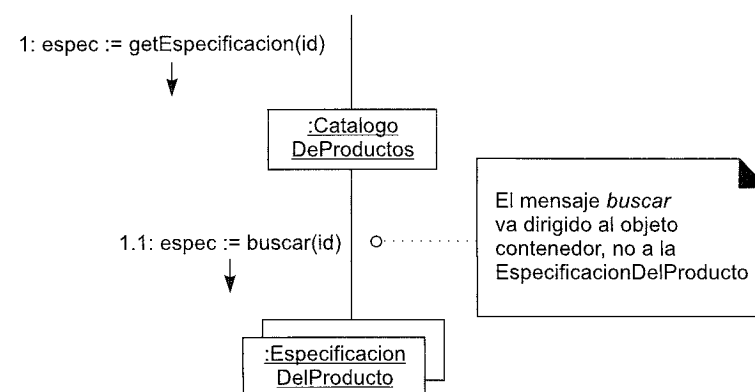


Figura 19.6. Mensaje a un multiobjeto.

Por tanto, el método *buscar* no forma parte de la clase *EspecificacionDelProducto* sino de la interfaz del multiobjeto. En consecuencia, no es correcto añadir el método *buscar* a la clase *EspecificacionDelProducto*.

Normalmente, estas interfaces o clases de contenedores/colecciones (como la interfaz *java.util.Map*) son elementos de las librerías predefinidas, y no es útil mostrar explícitamente estas clases en el DCD, porque añaden ruido, pero poca información nueva.

Nombres de los métodos—sintaxis dependiente del lenguaje

Algunos lenguajes, como Smalltalk, tienen una sintaxis que es muy diferente del formato UML básico de *nombreMetodo(listaParametros)*. Se recomienda que se utilice el formato UML básico, incluso si el lenguaje de implementación que se planea utilizar tiene una sintaxis diferente. Idealmente, la traducción debería tener lugar en el momento de la generación de código, en lugar de durante la creación de los diagramas de clase. Sin embargo, UML permite otra sintaxis para la especificación de los métodos.

Añadir más información sobre los tipos

Opcionalmente, todos los tipos de los atributos, parámetros de los métodos y los valores de retorno se podrían mostrar. La cuestión sobre si se muestra o no esta información se debe considerar en el siguiente contexto:

Se debería crear un DCD teniendo en cuenta los destinatarios.

- Si se está creando en una herramienta CASE con generación automática de código, son necesarios todos los detalles y de modo exhaustivo.
- Si se está creando para que lo lean los desarrolladores de software, los detalles exhaustivos de bajo nivel podrían afectar negativamente por el nivel de ruido.

Por ejemplo, ¿es necesario mostrar todos los parámetros y la información de sus tipos? Depende de lo obvia que sea la información para la audiencia a la que está destinada.

El diagrama de clases de diseño de la Figura 19.7 muestra más información sobre los tipos.

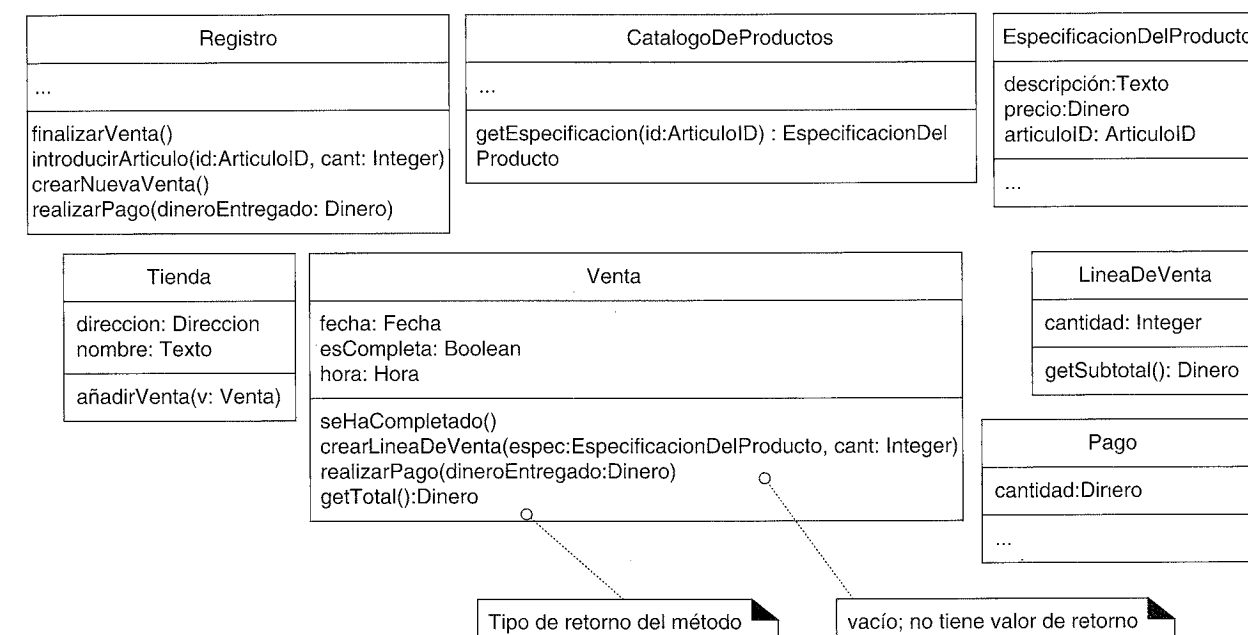


Figura 19.7. Añadir información acerca de los tipos.

Añadir asociaciones y navegabilidad

Cada extremo de asociación se denomina rol, y en los DCD el rol podría decorarse con una flecha de navegabilidad. La **navegabilidad** es una propiedad del rol que indica que es posible navegar unidireccionalmente a través de la asociación desde los objetos de la clase origen a la clase destino. La navegabilidad implica visibilidad —normalmente visibilidad de atributo (ver Figura 19.8)—.

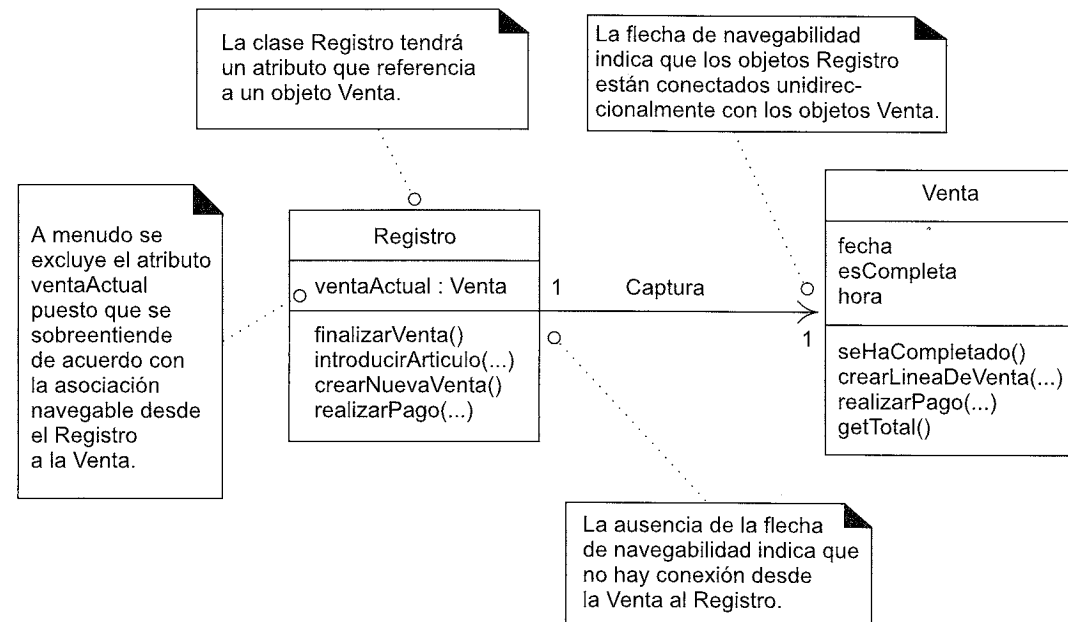


Figura 19.8. Representación de la navegabilidad o visibilidad de atributo.

La interpretación habitual de una asociación con una flecha de visibilidad es la visibilidad de atributo desde la clase origen hasta la clase destino. Durante la implementación en un lenguaje orientado a objetos por lo general se transforma en un atributo en la clase origen que hace referencia a una instancia de la clase destino. Por ejemplo, la clase *Registro* definirá un atributo que referencia a una instancia de *Venta*.

La mayoría, por no decir todas, las asociaciones en los DCD deberían adornarse con las flechas de navegabilidad necesarias.

En un DCD, se eligen las asociaciones de acuerdo a un criterio necesito-conocer orientado al software estricto —¿qué asociaciones se requieren para satisfacer la visibilidad y las necesidades de memoria actuales que se indican en los diagramas de interacción?—. Esto contrasta con las asociaciones en el Modelo del Dominio, que se podrían justificar por la intención de mejorar la comprensión del dominio del problema. Una vez más; vemos que existe una diferencia entre los objetivos del Modelo del Diseño y del Modelo del Dominio: uno es analítico, el otro una descripción de los componentes software.

La visibilidad y las asociaciones requeridas entre las clases se dan a conocer mediante los diagramas de interacción. A continuación, presentamos algunas situaciones comunes que sugieren la necesidad de definir una asociación con un adorno de visibilidad de A a B:

- A envía un mensaje a B.
- A crea una instancia de B.
- A necesita mantener una conexión a B.

Por ejemplo, a partir del diagrama de interacción de la Figura 19.9 que comienza con el mensaje *create* a la *Tienda*, y a partir del contexto más amplio de los otros diagramas

de interacción, se puede distinguir que probablemente la *Tienda* debería tener una conexión permanente con las instancias de *Registro* y el *CatalogoDeProductos* que crea. También es razonable que el *CatalogoDeProductos* necesite una conexión permanente con la colección de objetos *EspecificacionDelProducto* que crea. De hecho, muy a menudo el creador de otro objeto requiere una conexión permanente con él. Por tanto, las conexiones implícitas se presentarán como asociaciones en el diagrama de clases.

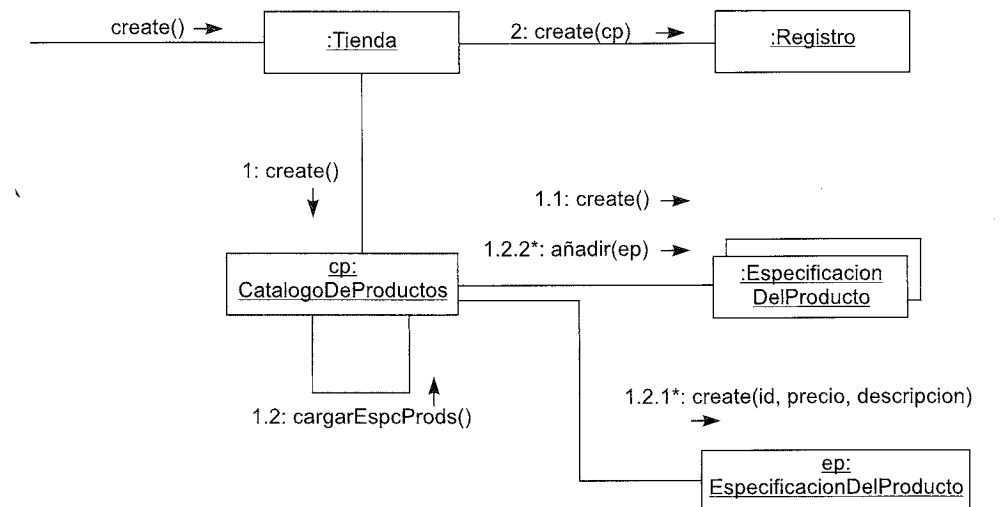


Figura 19.9. La navegabilidad se identifica a partir de los diagramas de interacción.

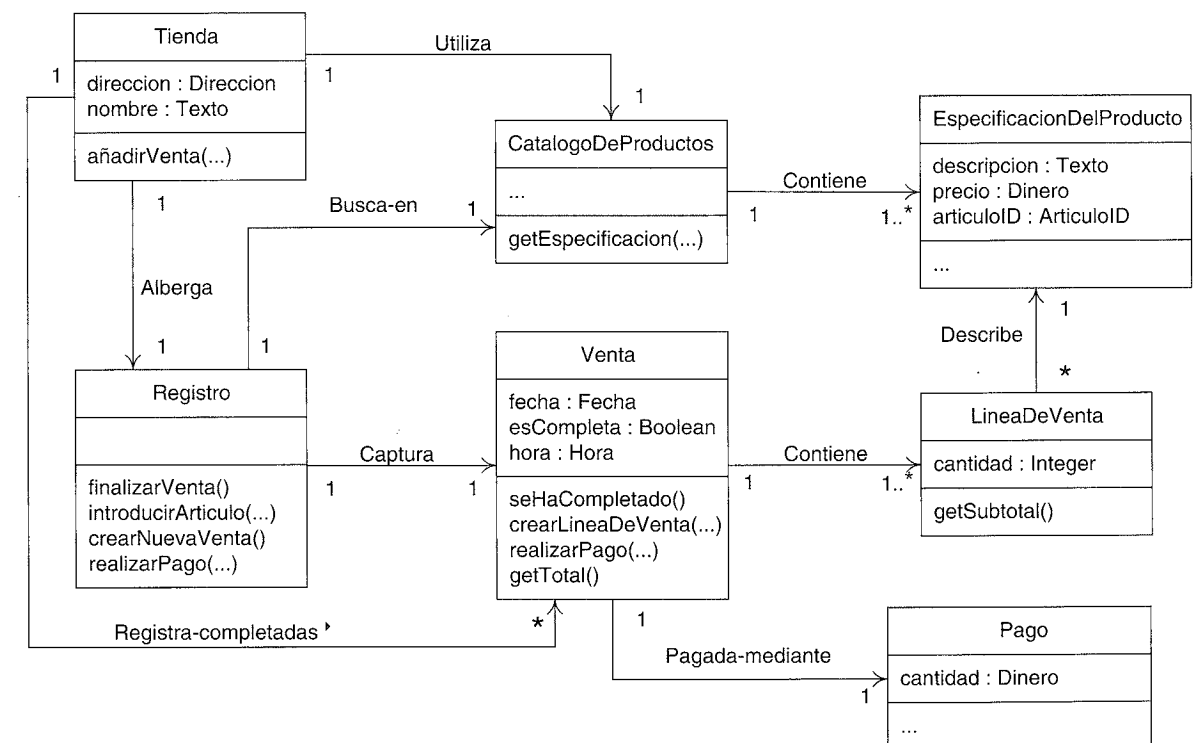


Figura 19.10. Asociaciones con adornos de navegabilidad.

Basado en el criterio anterior para las asociaciones y la navegabilidad, el análisis de todos los diagramas de interacción generados para la aplicación del PDV NuevaEra dará lugar a un diagrama de clases (ver Figura 19.10) con las siguientes asociaciones (se oculta la información exhaustiva sobre los tipos por claridad).

Nótese que éste no es exactamente el mismo conjunto de asociaciones que se generó para el diagrama de clases del Modelo del Dominio. Por ejemplo, en el modelo del dominio no existía la asociación *Busca-en* entre el *Registro* y el *CatalogoDeProductos* —no se pensó que fuera una relación importante y permanente en ese momento—. Pero durante la creación de los diagramas de interacción, se decidió que el objeto software *Registro* debería tener una conexión permanente con el *CatalogoDeProductos* software para buscar los objetos *EspecificacionDelProducto*.

Añadir las relaciones de dependencia

UML incluye una **relación de dependencia** general, que indica que un elemento (de cualquier tipo, como clases, casos de uso, etc.) tiene conocimiento de otro elemento. Se representa mediante una línea de flecha punteada. En los diagramas de clases la relación de dependencia es útil para describir la visibilidad entre clases que no es de atributo; en otras palabras, declaración de visibilidad de parámetro, local o global. En cambio, la vi-

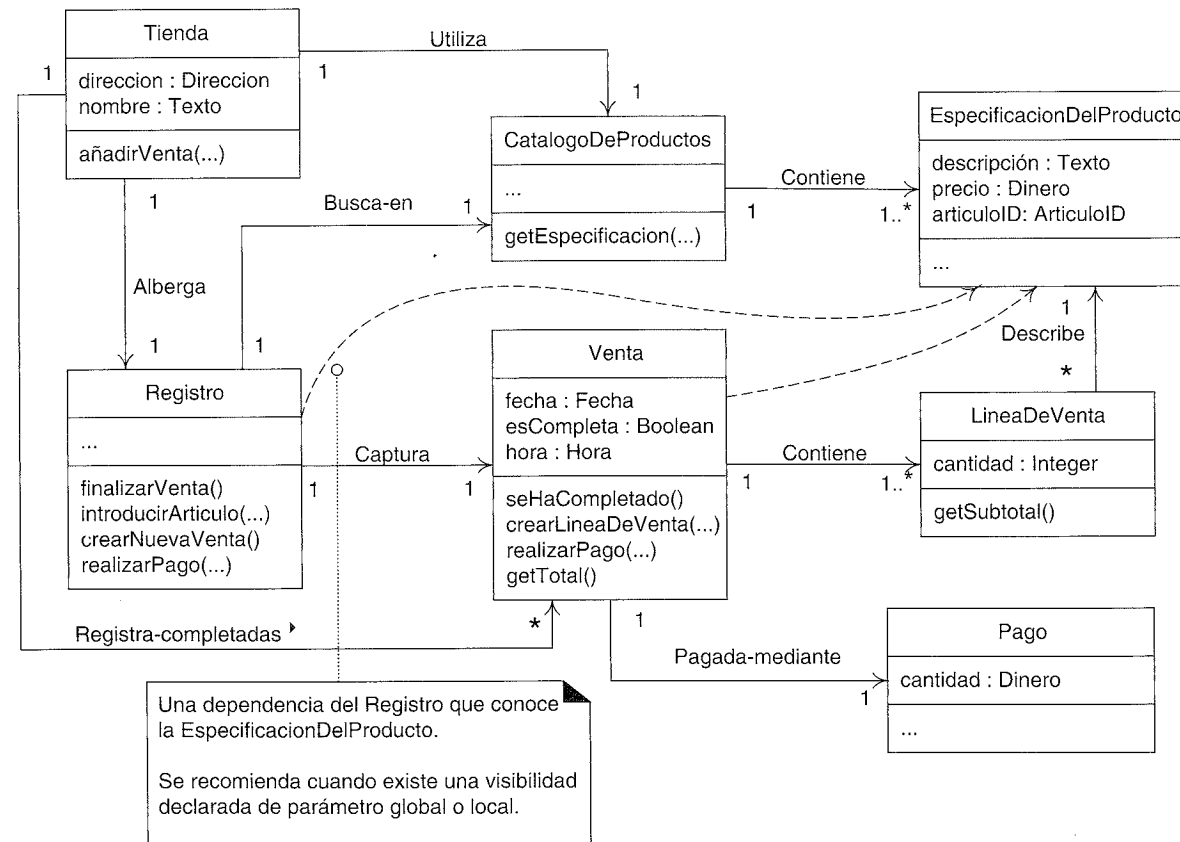


Figura 19.11. Relaciones de dependencia que indican una visibilidad que no es de atributo.

sibilidad de atributo simple se muestra mediante una línea de asociación ordinaria y una flecha de navegabilidad. Por ejemplo, el objeto software *Registro* recibe un objeto de retorno de tipo *EspecificacionDelProducto* a partir del mensaje de especificación que envía al *CatalogoDeProductos*. Por tanto, el *Registro* tiene una visibilidad a corto plazo, declarada localmente, de la *EspecificacionDelProducto*. Y una *Venta* recibe una *EspecificacionDelProducto* como parámetro en el mensaje *crearLineaDeVenta*; tiene visibilidad de parámetro de una especificación.

Estas visibilidades que no son de atributo podrían representarse con la línea de flecha punteada que indica una relación de dependencia (ver Figura 19.11). La curvatura de las líneas de dependencias no tiene ningún significado; se acomoda a la representación gráfica.

19.6. Notación para los detalles de los miembros

UML proporciona una notación variada para describir las características de los miembros de las clases e interfaces, como la visibilidad, valores iniciales, etcétera. En la Figura 19.12 se muestra un ejemplo.

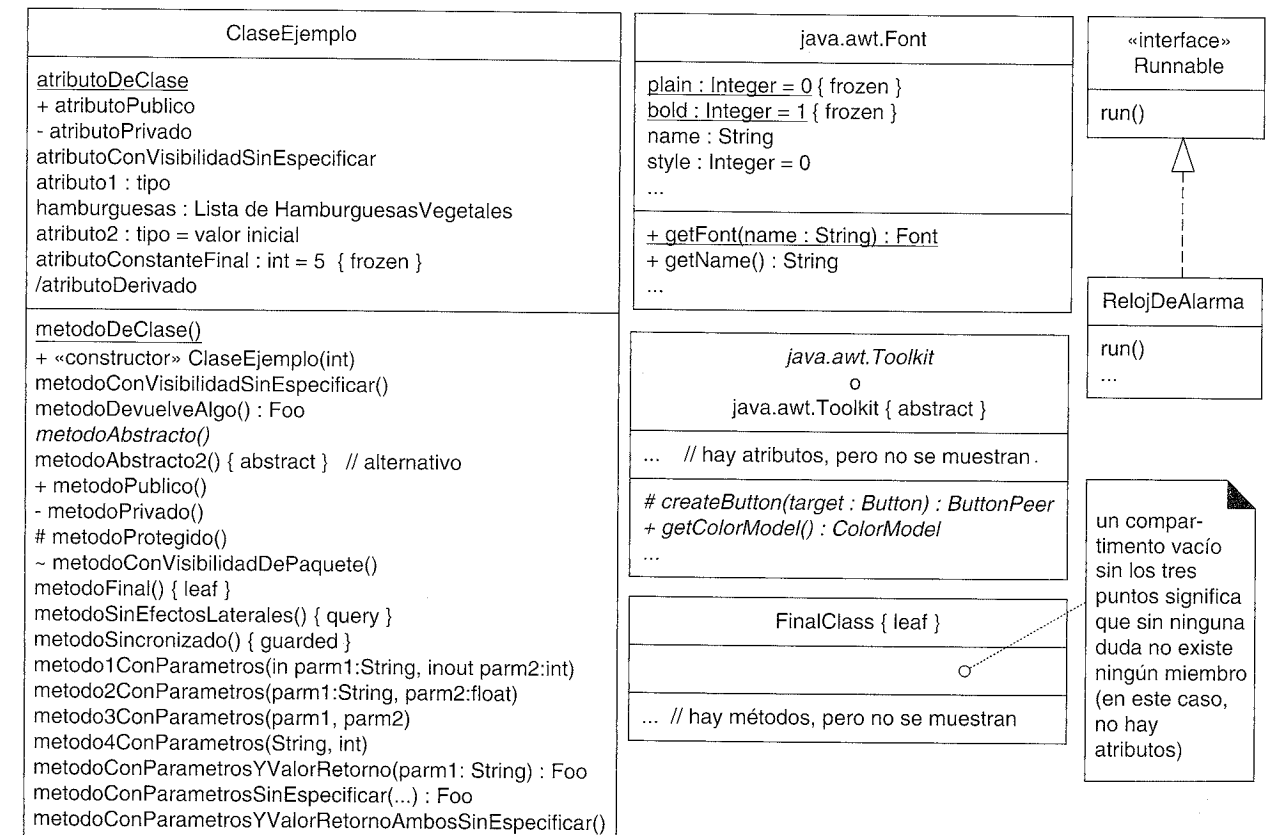


Figura 19.12. Detalles de la notación UML para algunos miembros de los diagramas de clases.

Visibilidad por defecto en UML

Si no se muestra explícitamente ningún marcador de visibilidad para un atributo o método, ¿cuál es el valor por defecto? Respuesta: no hay un valor por defecto. Si no se muestra nada, en UML significa “sin especificar”. Sin embargo, la convención común es asumir que los atributos son privados y los métodos públicos, a menos que se indique otra cosa.

La iteración actual del diagrama de clases de diseño del PDV NuevaEra (ver Figura 19.13) no tiene muchos detalles interesantes de los miembros; todos los atributos son privados y todos los métodos públicos.

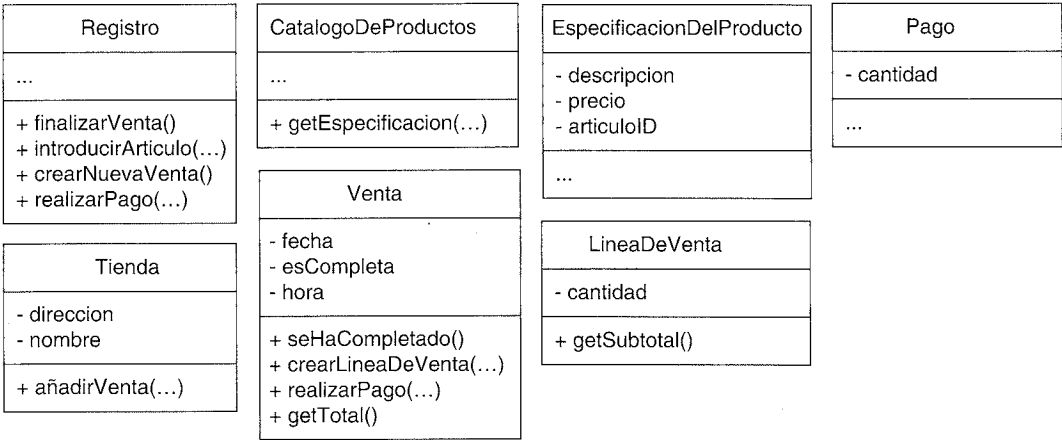


Figura 19.13. Detalles de los miembros del diagrama de clases del PDV.

Notación para el cuerpo de los métodos en los DCD (y los diagramas de interacción)

El cuerpo de un método se puede representar como se ilustra en la Figura 19.14 tanto en el DCD como en el diagrama de interacción.

19.7. DCD, dibujo y herramientas CASE

Las herramientas CASE pueden hacer ingeniería inversa (generar) de los DCD a partir del código fuente. En el Capítulo 35 sobre el dibujo y las herramientas CASE, se presentará una breve discusión sobre el contexto del proceso y la práctica de dibujar los DCD.

19.8. DCD en el UP

Los DCD forman parte de la realización de los casos de uso y, por tanto, miembros del Modelo de Diseño del UP.

Fases

Inicio: El Modelo de Diseño y los DCD normalmente no comenzarán hasta la elaboración porque comprenden decisiones de diseño, que son prematuras durante la fase de inicio.

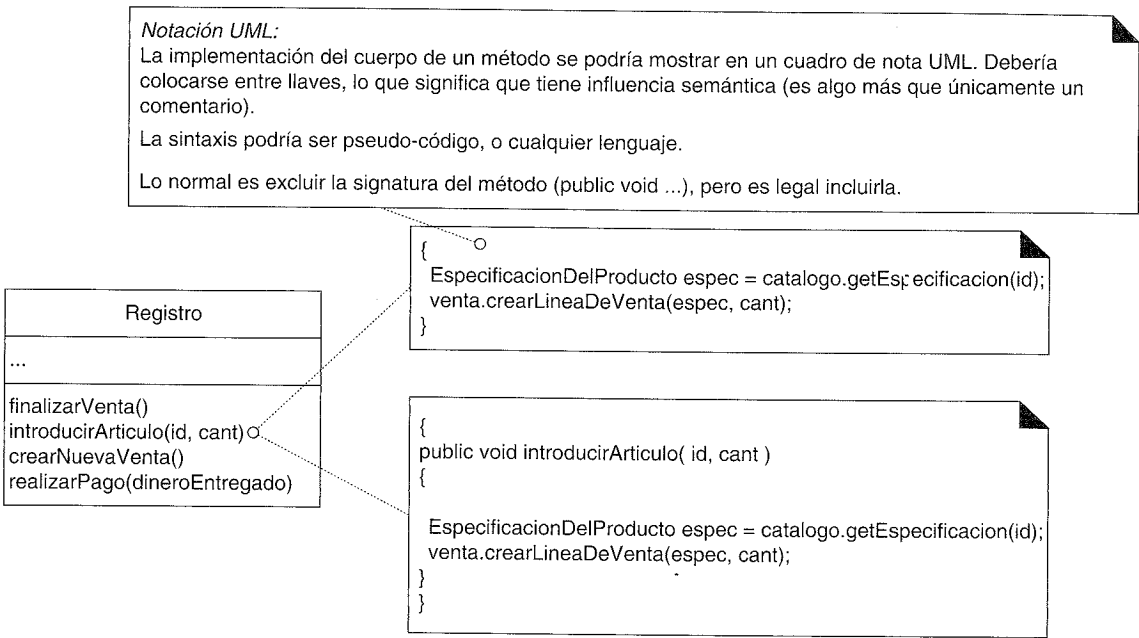


Figura 19.14. Notación para el cuerpo de los métodos.

Tabla 19.1. Muestra de los artefactos UP y evolución temporal. c-comenzar; r-refinar

Disciplina	Artefacto Iteración →	Inicio I1	Elab. E1...En	Const. C1...Cn	Trans. T1...T2
Modelado del Negocio	Modelo del Dominio		c		
Requisitos	Modelo de Casos de Uso (DSS)	c	r		
	Visión	c	r		
	Especificación Complementaria	c	r		
	Glosario	c	r		
Diseño	Modelo de Diseño		c	r	
	Documento de Arquitectura SW		c	r	
	Modelo de Datos		c	r	
Implementación	Modelo de Implementación		c	r	r
Gestión del Proyecto	Plan de Desarrollo SW	c	r	r	r
Pruebas	Modelo de Pruebas		c	r	
Entorno	Marco de Desarrollo	c	r		

Elaboración: Durante esta fase, los DCD acompañarán a los diagramas de interacción de las realizaciones de los casos de uso; podrían crearse para las clases del diseño más significativas desde el punto de vista de la arquitectura.

Nótese que las herramientas CASE pueden hacer ingeniería inversa (generar) de los DCD a partir del código fuente. Es recomendable generar los DCD de manera regular a partir del código fuente, para visualizar la estructura estática del sistema.

Construcción: Los DCD se continuarán generando a partir del código fuente como apoyo a la visualización de la estructura estática del sistema.

19.9. Artefactos del UP

La Figura 19.15 muestra la influencia entre los artefactos destacando los DCD.

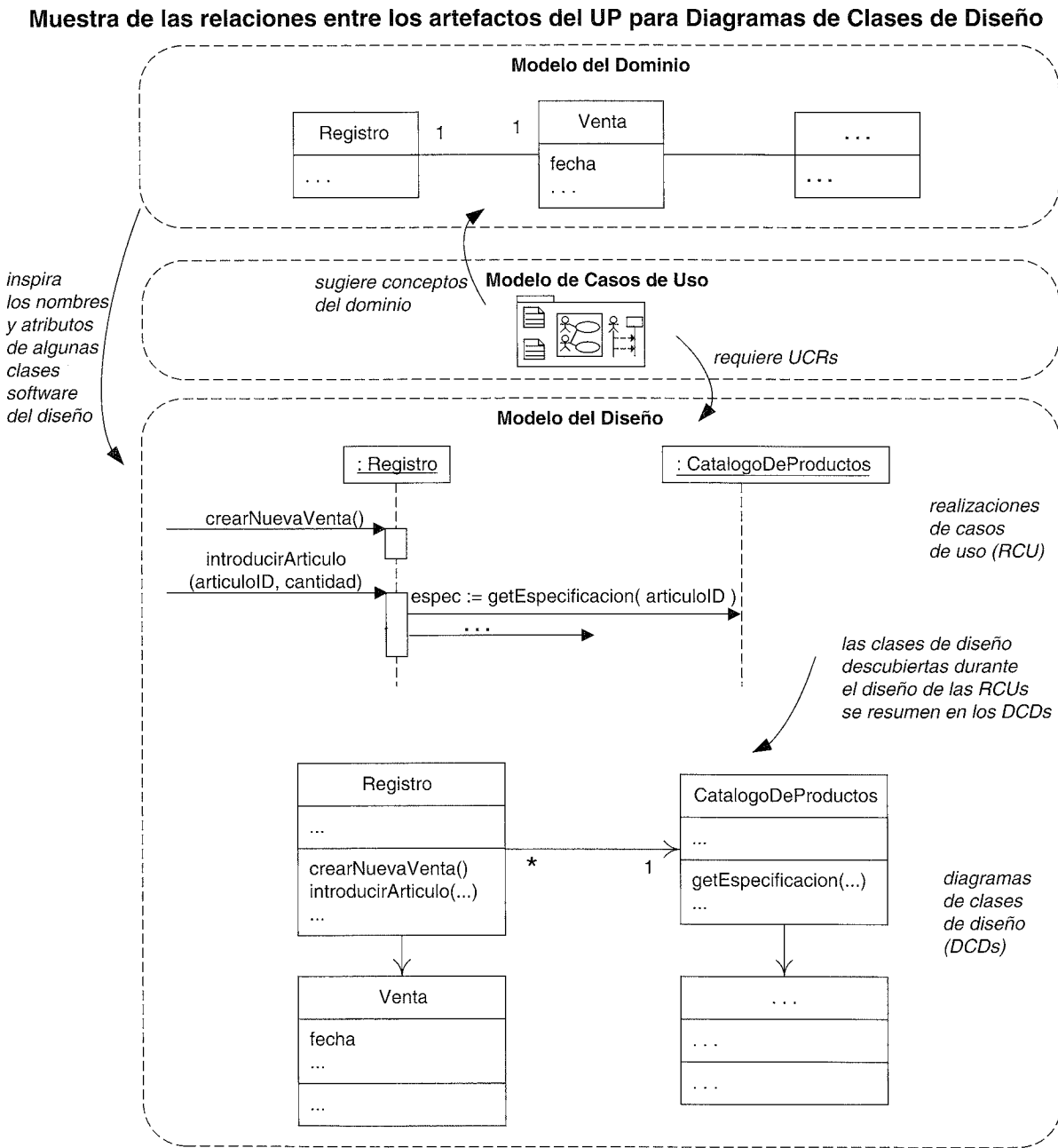


Figura 19.15. Muestra de la influencia entre los artefactos del UP.

MODELO DE IMPLEMENTACIÓN:
TRANSFORMACIÓN
DE LOS DISEÑOS EN CÓDIGO

Esté atento a los errores del programa anterior; sólo he demostrado que es correcto, pero nunca lo he usado.
Donald Knuth

Objetivos

- Transformar los artefactos del diseño en código en un lenguaje orientado a objetos.

Introducción

Después de completar los diagramas de interacción y los DCDs para la iteración actual de la aplicación NuevaEra, disponemos de suficientes detalles para generar el código de la capa del dominio de los objetos.

Los artefactos UML creados durante el trabajo de diseño —los diagramas de interacción y los DCDs— se utilizarán como entradas en el proceso de generación de código.

El UP define el Modelo de Implementación. Éste contiene los artefactos de implementación como el código fuente, las definiciones de bases de datos, las páginas JSP/XML/HTML, etcétera. Por tanto, el código que se va a crear en este capítulo forma parte del Modelo de Implementación.

Elección del lenguaje

Se utiliza Java para los ejemplos debido a su uso extendido y familiaridad. Sin embargo, la intención no es dar a entender que se recomienda Java en especial; C#, Visual Basic, C++ , Smalltalk, Python y muchos otros lenguajes son susceptibles de aplicar los prin-

cipios de diseño de objetos y la transformación a código que se presenta en este caso de estudio.

20.1. Programación y el proceso de desarrollo

El trabajo de diseño anterior no debería implicar que no exista prototipado o diseño durante la programación; las herramientas de desarrollo modernas proporcionan un entorno excelente para explorar rápidamente enfoques alternativos, y normalmente merece la pena elaborar algo (o incluso mucho) de diseño mientras se programa.

Sin embargo, algunos desarrolladores encuentran útil anticipar un poco mediante el modelado visual antes de programar, especialmente aquellos que se encuentran cómodos con el razonamiento visual o los lenguajes basados en diagramas.

Sugerencia

Para una iteración de dos semanas, considere dedicar por lo menos medio día próximo al comienzo de la iteración para llevar a cabo algo de trabajo de diseño mediante el modelado visual, antes de pasar a la programación. Utilice "herramientas" sencillas que soporten la realización rápida y creativa de diagramas, como una pizarra y una cámara digital. Si encuentra una herramienta para ingeniería del software asistida por ordenador (CASE, *computer-aid software engineering*) para UML que sea igualmente rápida, sencilla, y adecuada, excelente.

La creación de código en un lenguaje de programación orientado a objetos —como Java o C#— no forma parte del A/DOO; es un objetivo final. Los artefactos creados en el Modelo de Diseño del UP proporcionan parte de la información necesaria para generar el código.

Una ventaja del A/DOO y de la programación OO —cuando se utiliza con el UP— es que proporcionan una guía de principio a fin desde los requisitos hasta el código. Los distintos artefactos suministran información a los artefactos posteriores de manera útil y siguiendo una traza, culminando finalmente en una aplicación en ejecución. Esto no sugiere que el camino será fácil, o que se pueda seguir de manera mecánica —existen demasiadas variables—. Pero contar con una guía proporciona un punto de partida para experimentar y discutir.

Creatividad y cambio durante la implementación

Durante el trabajo de diseño se tomaron algunas decisiones y se llevó a cabo un trabajo creativo. Veremos a lo largo de la siguiente presentación que la generación de código —en este ejemplo— es un proceso de traducción relativamente mecánico.

Sin embargo, en general, el trabajo de programación no es una etapa de generación de código trivial, más bien lo contrario. En realidad, los resultados generados durante el diseño son un primer paso incompleto; durante la programación y pruebas, se realizarán innumerables cambios y se descubrirán y resolverán problemas complicados.

Si se hace bien, los artefactos del diseño proporcionarán un núcleo elástico que podrá extenderse con elegancia y robustez para satisfacer los nuevos problemas encon-

trados durante la programación. En consecuencia, espere y planifique para cambios y desviaciones del diseño durante la programación.

Cambios del código y el proceso iterativo

Una ventaja del proceso de desarrollo iterativo e incremental es que los resultados de la iteración anterior pueden proporcionar los datos para el comienzo de la siguiente iteración (ver Figura 20.1). Por tanto, los resultados del análisis y diseño actual se están refinando y enriqueciendo continuamente a partir del trabajo de implementación anterior. Por ejemplo, cuando el código de la iteración N se desvía del diseño de la iteración N (que lo hará inevitablemente), el diseño final que es la base de la implementación puede ser la entrada para los modelos de análisis y diseño de la iteración N+1.

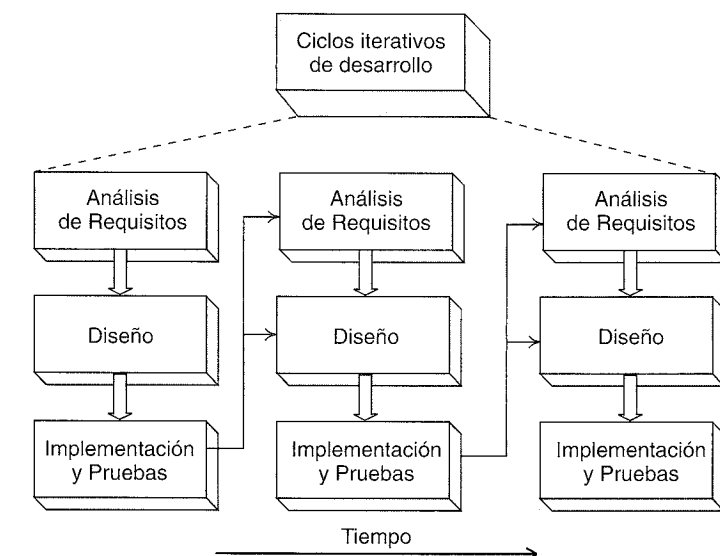


Figura 20.1. La implementación en una iteración influye en el diseño posterior.

Una de las primeras actividades en una iteración es sincronizar los diagramas de diseño; los primeros diagramas de la iteración N no corresponderán con el código final de esa misma iteración, y necesitan sincronizarse antes de que se extiendan con nuevos resultados del diseño.

Cambios en el código, herramientas CASE, e ingeniería inversa

Es deseable que los diagramas generados durante el diseño se actualicen de manera semi-automática para reflejar los cambios en el trabajo de codificación siguiente. Generalmente esto debería hacerse con una herramienta CASE que puede leer el código fuente y generar automáticamente, por ejemplo, diagramas de paquetes, clases y secuencia. Este es un aspecto de la **ingeniería inversa** —la actividad de generar diagramas a partir del código fuente (o a veces, ejecutable)—.

20.2. Transformación de los diseños en código

La implementación en un lenguaje orientado a objetos requiere la escritura de código fuente para:

- Las definiciones de las clases e interfaces.
- Las definiciones de los métodos.

Las secciones siguientes presentarán la generación en Java (como un caso típico).

20.3. Creación de las definiciones de las clases a partir de los DCDs

Como mínimo, los DCDs describen los nombres de las clases o interfaces, las superclases, signature de los métodos y los atributos simples de una clase. Esto es suficiente para crear una definición de clase básica en un lenguaje de programación orientado a objetos. Una discusión posterior explorará la incorporación de la información acerca de las interfaces y ámbito de nombres (o paquetes), entre otros detalles.

Definición de una clase con métodos y atributos simples

A partir del DCD, la transformación de las definiciones de los atributos básicas (campos de instancia simples de Java) y las signatures de los métodos de la *LineaDeVenta* a la definición Java es directa, como se muestra en la Figura 20.2.

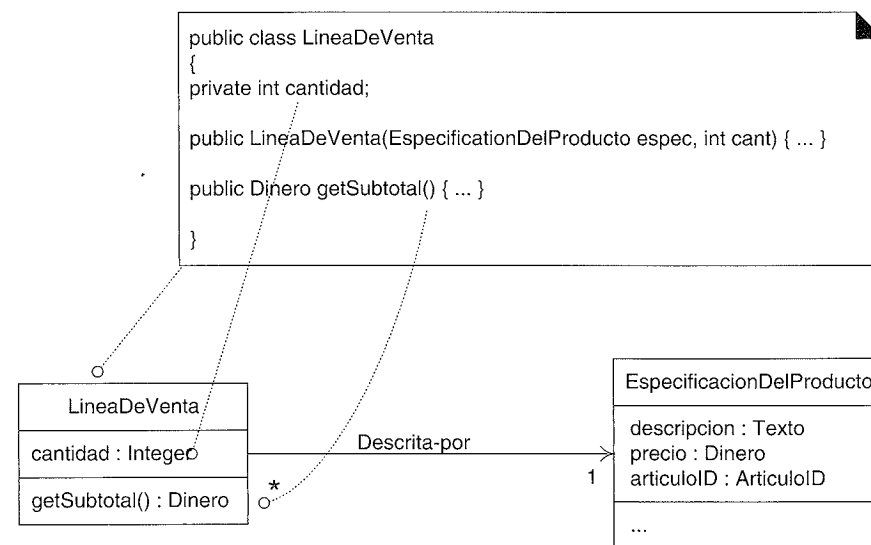


Figura 20.2. LineaDeVenta en Java.

Obsérvese que se ha incorporado en el código fuente el constructor Java *LineaDeVenta(...)*. Se deriva del envío del mensaje *create(espec, ctd)* a la *LineaDeVenta* en el diagrama de interacción *introducirArticulo*. Esto indica, en Java, que se requiere un constructor que soporte estos parámetros. A menudo se excluye de los diagramas de clases el método *create* puesto que es común que aparezca y tiene múltiples interpretaciones, dependiendo del lenguaje que se vaya a utilizar.

Añadir atributos de referencia

Un **atributo de referencia** es un atributo que referencia a otro objeto complejo, no a un tipo primitivo como una cadena de texto, un número, etcétera.

Los atributos de referencia de una clase se deducen de las asociaciones y la navegabilidad de un diagrama de clases.

Por ejemplo, una *LineaDeVenta* tiene una asociación con una *EspecificacionDelProducto*, con navegabilidad hacia ella. Es habitual que esto se interprete como un atributo de referencia en la clase *LineaDeVenta* que hace referencia a la instancia de *EspecificacionDelProducto* (ver Figura 20.3).

En Java, esto significa que se recomienda un campo de instancia que referencia a una instancia de *EspecificacionDelProducto*.

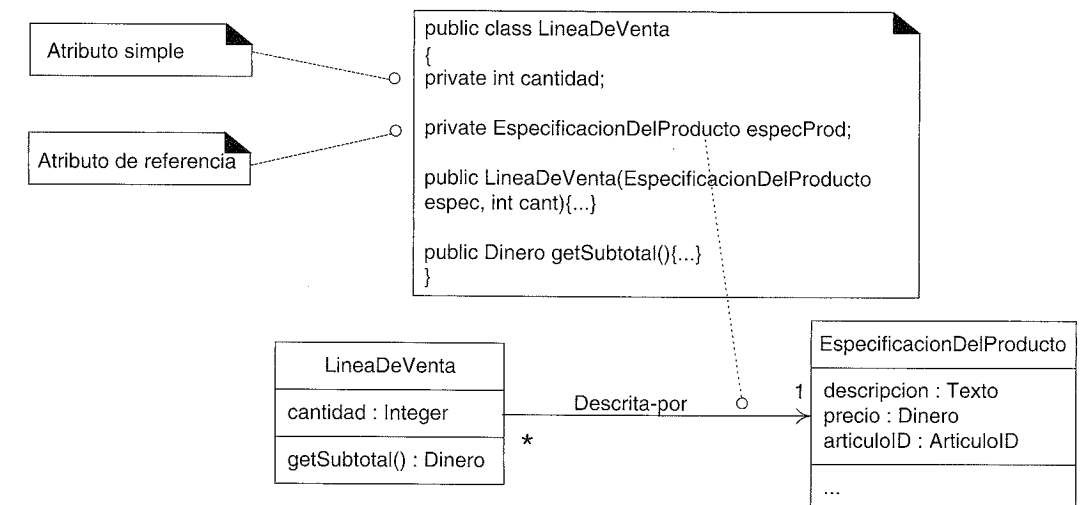


Figura 20.3. Inclusión de atributos de referencia.

Obsérvese que los atributos de referencia de una clase a menudo están implícitos, en lugar de explícitos, en un DCD.

Por ejemplo, aunque hemos añadido un campo de instancia en la definición Java de *LineaDeVenta* para que apunte a una *EspecificacionDelProducto*, no se declara explícitamente como un atributo en la sección de atributos del rectángulo de la clase. Existe una visibilidad de atributo *recomendada* —indicada por la asociación y navegabilidad— que se define explícitamente mediante un atributo durante la fase de generación de código.

Atributos de referencia y los nombres de los roles

La siguiente iteración estudiará el concepto de los nombres de los roles en los diagramas de estructura estáticos. Cada extremo de asociación se denomina rol. Brevemente, un **nombre de rol** es un nombre que identifica al rol y, a menudo, proporciona algo del contexto semántico acerca de la naturaleza del rol.

Si el nombre de un rol está presente en un diagrama de clases, utilícelo como base para el nombre del atributo de referencia durante la generación de código, como se muestra en la Figura 20.4.

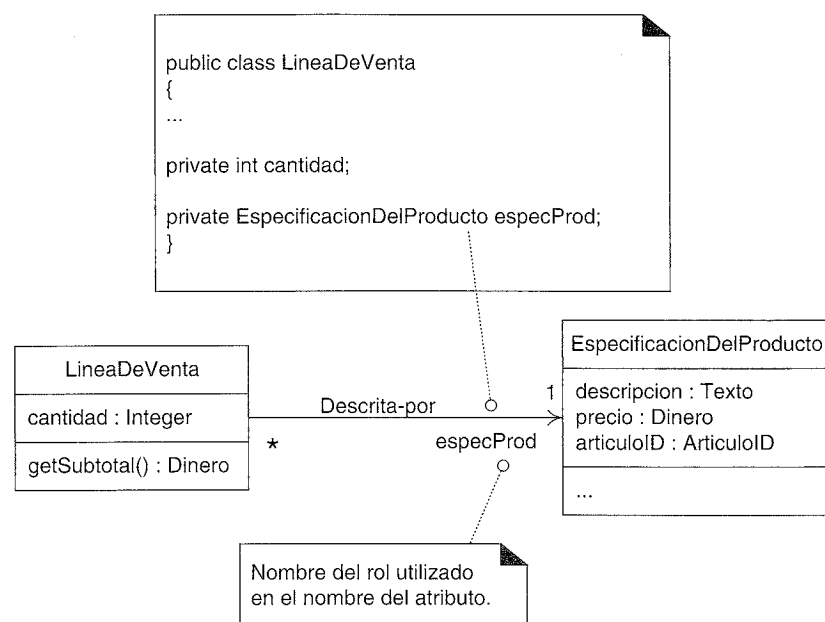


Figura 20.4. Los nombres de los roles podrían utilizarse para generar los nombres de las variables de instancia.

Transformación de los atributos

La clase *Venta* ilustra que en algunos casos uno debe considerar la transformación de los atributos desde el diseño al código en lenguajes diferentes. La Figura 20.5 muestra el problema y su solución.

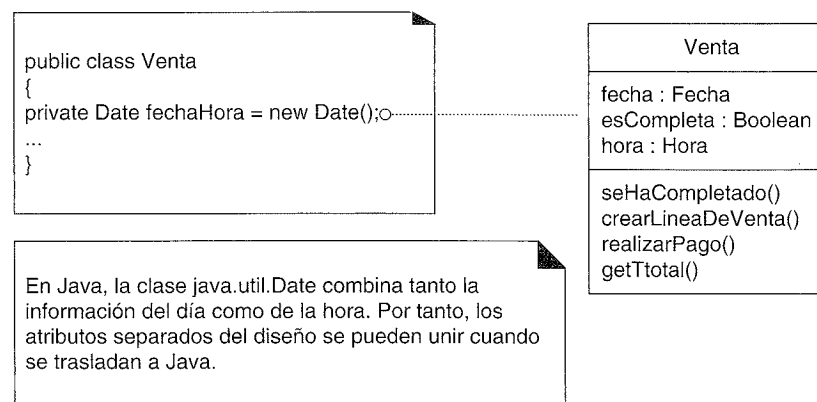


Figura 20.5. Transformación de la fecha y la hora en Java.

20.4. Creación de métodos a partir de los diagramas de interacción

Un diagrama de interacción muestra los mensajes que se envían como respuesta a la invocación de un método. La secuencia de estos mensajes se traduce en una serie de sentencias en la definición del método. El diagrama de interacción *introducirArticulo* de la Figura 20.6 ilustra la definición Java del método *introducirArticulo*.

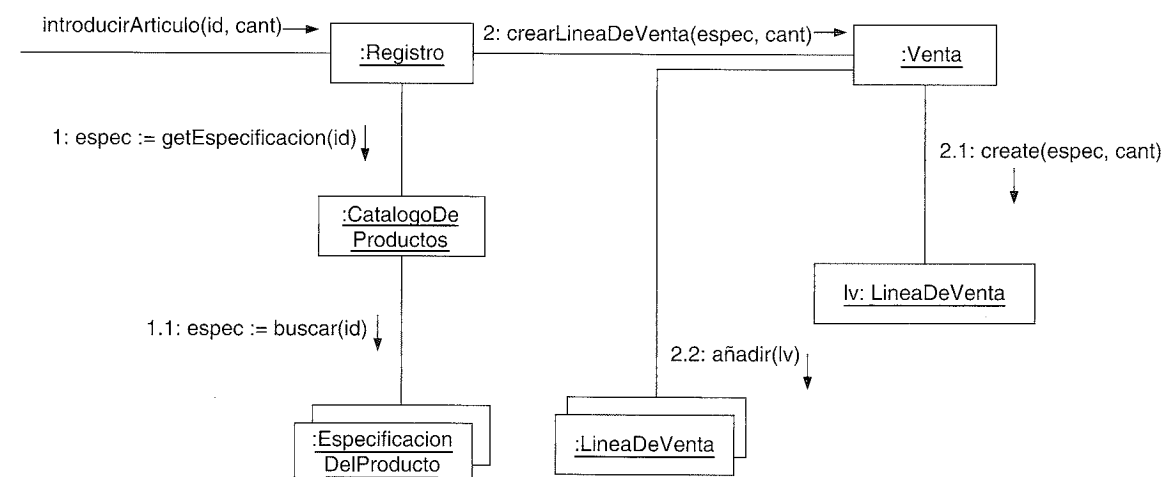


Figura 20.6. El diagrama de interacción de introducirArticulo.

En este ejemplo, se utilizará la clase *Registro*. La Figura 20.7 muestra una definición en Java.

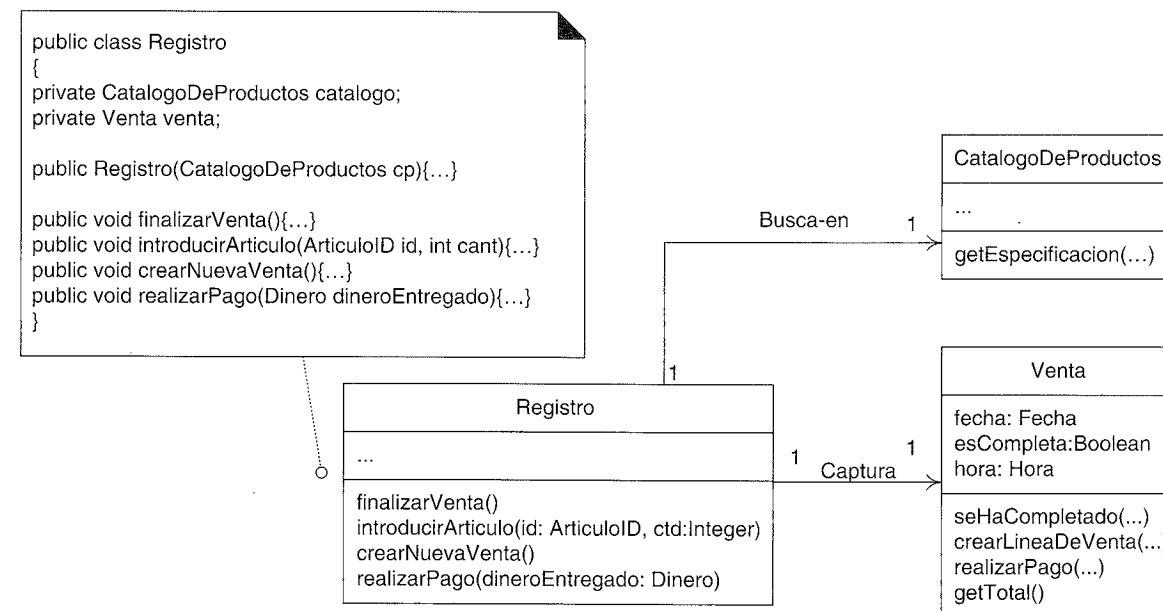


Figura 20.7. La clase Registro.

El método Registro--introducirArticulo

Se envía el mensaje *introducirArticulo* a una instancia de *Registro*; por tanto, se define el método *introducirArticulo* en la clase *Registro*.

```
public void introducirArticulo(ArticuloID articuloID, int ctd)
```

Mensaje 1: Se envía el mensaje *getEspecificacion* al *CatalogoDeProductos* para recuperar una *EspecificacionDelProducto*.

```
EspecificacionDelProducto espec =  
catalogo.getEspecificacion(articuloID);
```

Mensaje 2: Se envía el mensaje *crearNuevaVenta* a la *Venta*.

```
venta.crearLineaDeVenta(espec, ctd);
```

Resumiendo, cada mensaje de la secuencia del método, que se muestra en el diagrama de interacción, se transforma en una sentencia del método Java.

La Figura 20.8 muestra el método completo *introducirArticulo* y su relación con el diagrama de interacción.

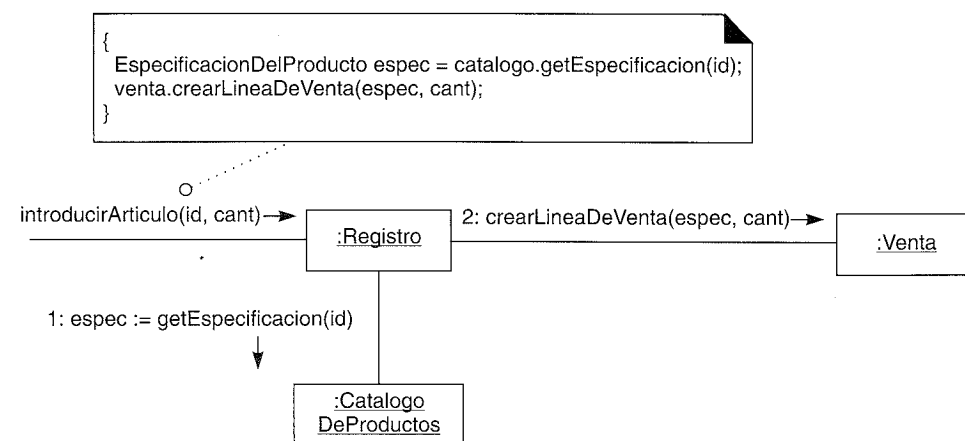


Figura 20.8. El método introducirArticulo.

20.5. Clases contenedoras/colecciones en el código

A menudo es necesario que un objeto mantenga la visibilidad a un grupo de otros objetos; normalmente esta necesidad es evidente a partir del valor de la multiplicidad en el diagrama de clases —podría ser mayor que uno—. Por ejemplo, una *Venta* debe mantener la visibilidad a un grupo de instancias de *LineaDeVenta*, como se muestra en la Figura 20.9.

En los lenguajes de programación OO, con frecuencia se implementan estas relaciones introduciendo un contenedor o colección intermedia. La clase del lado del uno de-

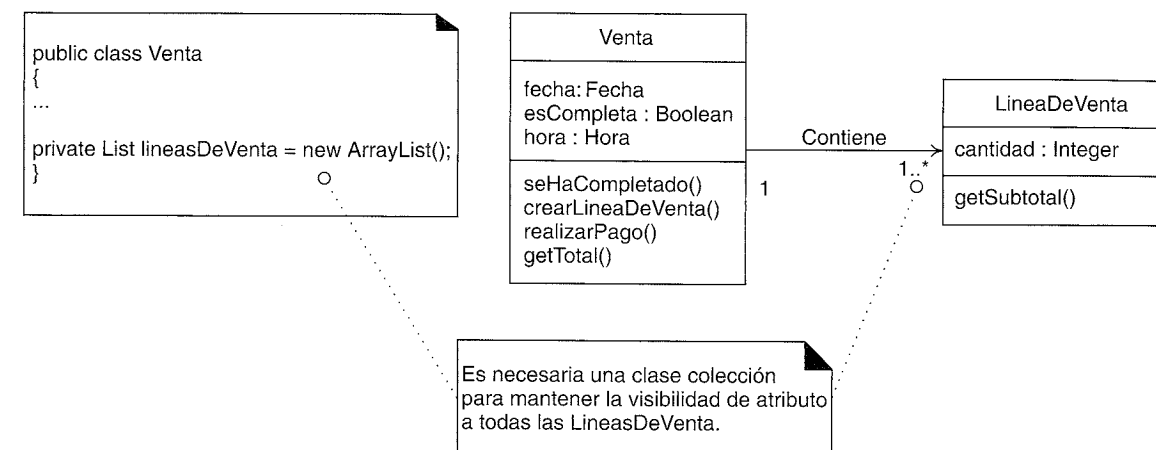


Figura 20.9. Incorporación de una colección.

fine un atributo de referencia que apunta a la instancia del contenedor/colección, que contiene instancias de la clase del lado de muchos.

Por ejemplo, las librerías de Java contienen las clases de colección como *ArrayList* y *HashMap*, que implementan las interfaces de *List* y *Map*, respectivamente. Utilizando *ArrayList*, la clase *Venta* puede definir un atributo que mantenga una lista ordenada de las instancias de *LineaDeVenta*.

Por supuesto, los requisitos influyen en la elección de la clase colección; las búsquedas por claves requieren el uso de un *Map*, una lista ordenada creciente requiere una *List*, etcétera.

20.6. Manejo de excepciones y de errores

Hasta el momento se ha ignorado el manejo de las excepciones en el desarrollo de una solución. Esto se hizo a propósito para centrarnos en las cuestiones básicas de la asignación de responsabilidades y el diseño de objetos. Sin embargo, en el desarrollo de la aplicación, es aconsejable tener en cuenta el manejo de las excepciones durante el trabajo de diseño y, por supuesto, durante la implementación.

Brevemente, en UML, las excepciones se representan como mensajes asíncronos en los diagramas de interacción. Esto se estudiará en el Capítulo 33.

20.7. Definición del método Venta--crearLineaDeVenta

Como último ejemplo, también se puede escribir el método *crearLineaDeVenta* de la clase *Venta* inspeccionando el diagrama de colaboración *introducirArticulo*. La Figura 20.10 muestra una versión abreviada del diagrama de interacción, con el método Java acompañante.

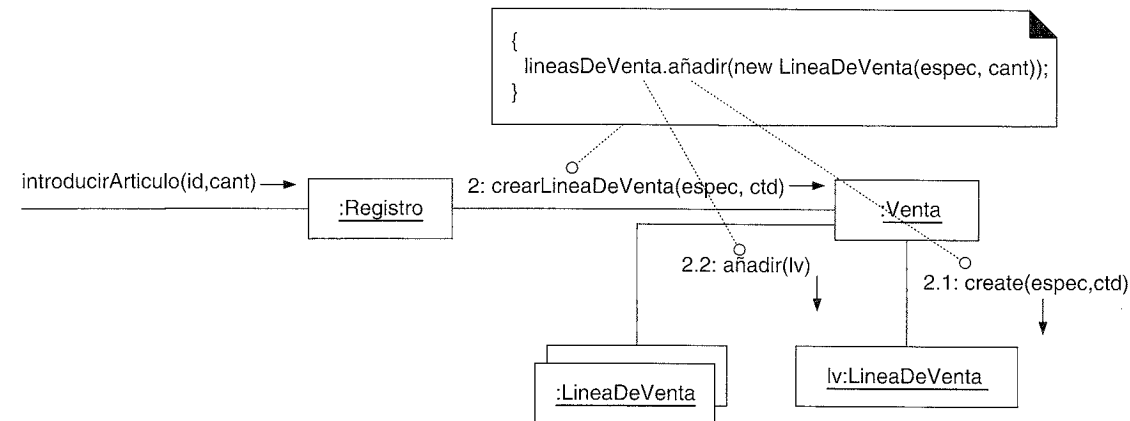


Figura 20.10. Método Venta--crearLineaDeVenta.

20.8. Orden de implementación

Es necesario implementar las clases (e idealmente, hacer las pruebas de unidad completamente) desde la menos a la más acoplada (ver Figura 20.11). Por ejemplo, las posibles primeras clases que podríamos implementar son *Pago* o *EspecificacionDelProducto*; a continuación las clases que sólo dependen de la implementación anterior —*CatalogoDeProductos* o *LineaDeVenta*—.

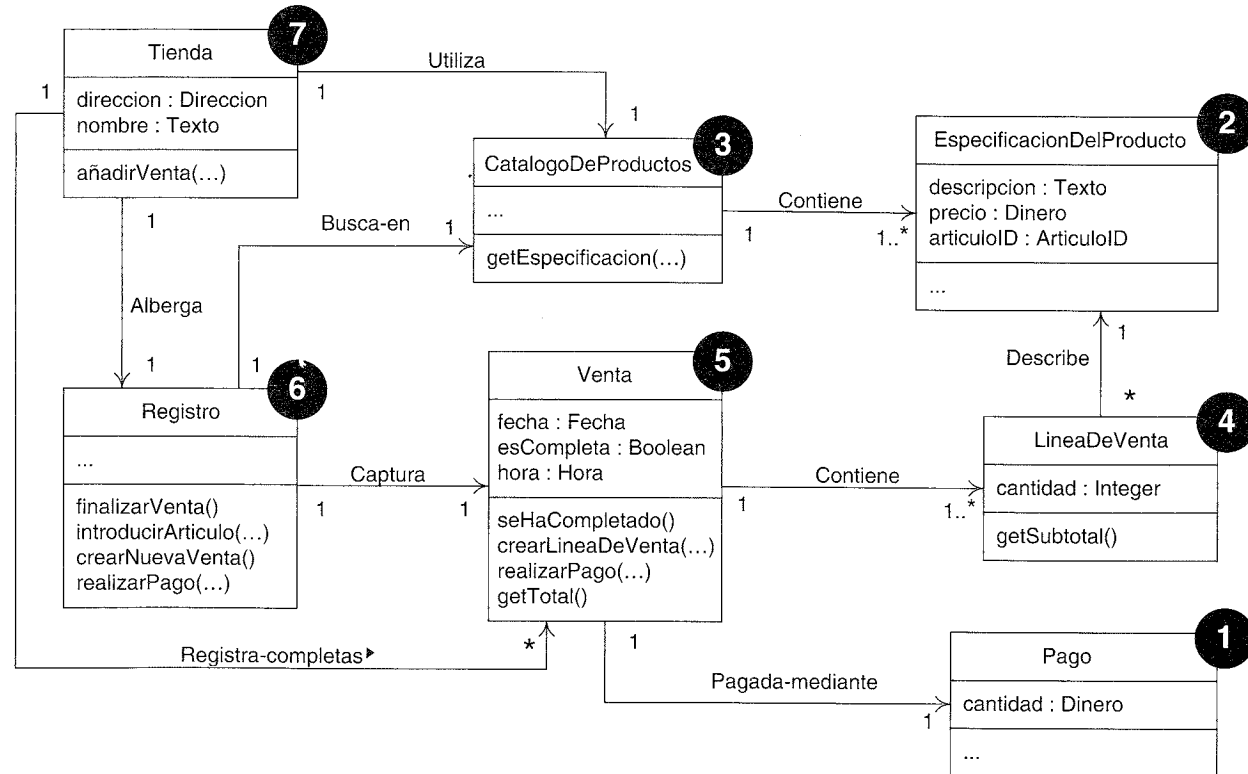


Figura 20.11. Posible orden para la implementación y prueba de las clases.

20.9. Programar probando primero

Una práctica excelente promovida por el método de Programación Extrema (XP, *eXtreme Programming*) [Beck00], y aplicable al UP (como son la mayoría de las prácticas de XP), es **programar probando primero**. En esta práctica, se escribe el código de las pruebas de unidad *antes* del código que se va a probar, y los desarrolladores escriben el código de las pruebas de unidad para *todo* el código de producción. La secuencia básica es escribir un poco del código de prueba, luego escribir un poco del código de producción, hacer que pase las pruebas, y entonces escribir más código de prueba, y así sucesivamente.

Entre las ventajas encontramos:

- **Se escriben realmente las pruebas de unidad:** La naturaleza humana (o al menos la de los programadores) es de tal manera que es muy común evitar la escritura de las pruebas de unidad, si se dejó como una idea adicional.
- **Satisfacción de los programadores:** Si un desarrollador escribe el código de producción, depura de manera informal, y entonces como ocurrencia tardía añade pruebas de unidad, no se siente muy satisfecho. Sin embargo, si se escriben primero las pruebas, y entonces se crea y refina el código de producción para que pase las pruebas, existe un sentimiento de logro —de pasar una prueba—. No se puede ignorar el aspecto psicológico del desarrollo —la programación es un esfuerzo humano—.
- **Aclaración de las interfaces y el comportamiento:** A menudo, la interfaz pública exacta y el comportamiento de una clase no están perfectamente claros hasta que se implementan. Escribiendo las pruebas de unidad primero, uno aclara el diseño de la clase.
- **Verificación demostrable:** Obviamente, el contar con cientos o miles de pruebas de unidad proporciona una verificación significativa de la corrección.
- **La confianza en cambiar cosas:** Si programamos probando primero, tendremos de cientos o miles de pruebas de unidad, y una clase de pruebas de unidad por cada clase de producción. Cuando un desarrollador necesita cambiar el código existente —escrito por ellos mismos o por otros— existe un conjunto de pruebas de unidad que pueden ejecutarse, proporcionando una retroalimentación inmediata sobre si el cambio causó un error.

Como ejemplo, JUnit (www.junit.org) es un marco de pruebas de unidad para Java gratuito, simple y popular. Suponga que estamos utilizando JUnit y seguimos la estrategia de programar probando primero, para crear la clase *Venta*. *Antes* de programar la clase *Venta*, escribimos un método de prueba de unidad en la clase *PruebaVenta* que hace lo siguiente:

- Crea una nueva venta.
- Le añade algunas líneas de venta.
- Solicita el total y comprueba si tiene el valor esperado.

Por ejemplo,

```

public class PruebaVenta extends TestCase
{
    //...
    public void pruebaTotal()
    {
        //inicializa la prueba
        Dinero total = new Dinero(7.5);
        Dinero precio = new Dinero (2.5);
        ArtículoID id = new ArtículoID (1);
        EspecificacionDelProducto espec;
        espec = new EspecificacionDelProducto(id, precio, "producto 1");
        Venta venta = new Venta();

        //añade los artículos
        venta.crearLineaDeVenta(espec, 1);
        venta.crearLineaDeVenta(espec, 2);

        //Comprobar que el total es 7.5
        assertEquals(venta.getTotal(), total);
    }
}

```

Sólo después de crear esta clase *PruebaVenta* será cuando escribamos la clase *Venta* que pase esta prueba. Sin embargo, no es necesario escribir todos los métodos de prueba de antemano. Un desarrollador escribe un método de prueba, después el código de producción que la satisface, a continuación otro método de prueba, y así sucesivamente.

20.10. Resumen de la transformación de los diseños en código

El proceso de traducción de los DCD a las definiciones de las clases, y de los diagramas de interacción en los métodos, es relativamente directo. Durante el trabajo de programación todavía hay cabida para tomar decisiones, realizar cambios en el diseño y explorar, pero algunas de las grandes ideas del diseño se tuvieron en cuenta antes de la programación.

20.11. Introducción a la solución del programa

Esta sección presenta una muestra de la solución del programa de la capa de los objetos del dominio en Java para esta iteración. La generación del código se deriva casi por completo de los diagramas de clases del diseño y los diagramas de interacción que se definieron en el trabajo de diseño, y se basa en los principios de transformación de los diseños a código que se han estudiado previamente.

Lo más importante de este listado es mostrar que se traducen los artefactos de diseño a una base de código. Este código define un caso sencillo; la intención no es ilustrar un programa Java robusto y totalmente desarrollado con sincronización, manejo de excepciones, etcétera.

Clase Pago

```

public class Pago
{
    private Dinero cantidad;
    public Pago(Dinero dineroEntregado){ cantidad = dineroEntregado; }
    public Dinero getCantidad() { return cantidad; }
}

```

Clase CatalogoDeProductos

```

public class CatalogoDeProductos
{
    private Map especificacionesDeProductos = new HashMap();

    public CatalogoDeProductos()
    {
        // datos de ejemplo
        ArtículoID id1 = new ArtículoID( 100 );
        ArtículoID id2 = new ArtículoID( 200 );
        Dinero precio = new Dinero( 3 );

        EspecificacionDelProducto ep;
        ep = new EspecificacionDelProducto( id1, precio, "producto 1");
        especificacionesDeProductos.put( id1, ep );
        ep = new EspecificacionDelProducto( id2, precio, "producto 2");
        especificacionesDeProductos.put( id2, ep );
    }

    public EspecificacionDelProducto getEspecificacion (ArtículoID id)
    {
        return (EspecificacionDelProducto)especificacionesDeProductos.get(id);
    }
}

```

Clase Registro

```

public class Registro{
{
    private CatalogoDeProductos catalogo;
    private Venta venta;

    public Registro(CatalogoDeProductos catalogo)
    {
        this.catalogo = catalogo;
    }

    public void finalizarVenta()
    {
        venta.seHaCompletado();
    }

    public void introducirArticulo(ArtículoID id, int cantidad)
    {
        EspecificacionDelProducto espec = catalogo.getEspecificacion(id);
    }
}

```

```

    venta.crearLineaDeVenta( espec, cantidad);
    }

    public void crearNuevaVenta()
    {
        venta = new Venta();
    }

    public void realizarPago(Dinero dineroEntregado)
    {
        venta.realizarPago(dineroEntregado);
    }
}

```

Clase EspecificacionDelProducto

```

public class EspecificacionDelProducto
{
    private ArtículoID id;
    private Dinero precio;
    private String descripcion;

    public EspecificacionDelProducto
        (ArtículoID id, Dinero precio, String descripcion)
    {
        this.id = id;
        this.precio = precio;
        this.descripcion = descripcion;
    }

    public ArtículoID getArtículoID() { return id; }

    public Dinero getPrecio() { return precio; }

    public String getDescripcion() { return descripcion; }
}

```

Clase Venta

```

public class Venta
{
    private List lineasDeVenta = new ArrayList();
    private Date fecha = new Date();
    private boolean esCompleta() = false;
    private Pago pago;

    public Dinero getDevolucion()
    {
        return pago.getCantidad().minus( getTotal() );
    }

    public void seHaCompletado(){ esCompleta = true; }

    public boolean esCompleta(){ return esCompleta;}
}

```

```

public void crearLineaDeVenta
    (EspecificacionDelProducto espec, int cantidad)
{
    lineasDeVenta.add( new LineaDeVenta(espec, cantidad) );
}

public Dinero getTotal()
{
    Dinero total = new Dinero();
    Iterator i = lineasDeVenta.iterator();
    while (i.hasNext())
    {
        LineaDeVenta ldv = (LineaDeVenta)i.next();
        total.add(ldv.getSubtotal() );
    }
    return total;
}

public void realizarPago(Dinero dineroEntregado)
{
    pago = new Pago( dineroEntregado);
}
}

```

Clase LineaDeVenta

```

public class LineaDeVenta
{
    private int cantidad;
    private EspecificacionDelProducto especProducto;

    public LineaDeVenta (EspecificacionDelProducto espec, int cantidad)
    {
        this.especProducto = espec;
        this.cantidad = cantidad;
    }

    public Dinero getSubtotal()
    {
        return especProducto.getPrecio().times( cantidad );
    }
}

```

Clase Tienda

```

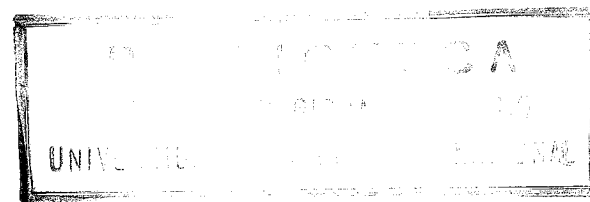
public class Tienda
{
    private CatalogoDeProductos catalogo = new CatalogoDeProductos();
    private Registro registro = new Registro(catalogo);

    public Registro getRegistro() { return registro; }
}

```

Parte 4

ELABORACIÓN EN LA ITERACIÓN 2



LA ITERACIÓN 2 Y SUS REQUISITOS

21.1. Énfasis de la Iteración 2: diseño de objetos y patrones

Los capítulos de la fase de concepción y los de la Iteración 1 de la fase de elaboración han hecho hincapié en un conjunto de técnicas fundamentales del análisis y diseño de objetos, con el objeto de compartir información sobre una amplia gama de pasos comunes en la construcción de sistemas de objetos.

En esta iteración, el caso de estudio se centrará en:

- El diseño de objetos esencial.
- El uso de patrones para crear un diseño sólido.
- La aplicación de UML para visualizar los modelos.

Estos son objetivos básicos del libro y habilidades necesarias.

Hay una discusión mínima acerca del análisis de requisitos o el modelado del dominio, y la explicación del diseño es más concisa, ahora que (en la Iteración 1) ya se ha presentado una explicación detallada sobre las bases de cómo pensar en objetos.

Por supuesto, en esta iteración tendrían lugar muchas otras actividades de análisis, diseño e implementación, pero se les presta menos atención para centrarnos en proporcionar información sobre cómo hacer el diseño de objetos.

21.2. De la Iteración 1 a la 2

Cuando termina la Iteración 1, se debería cumplir lo siguiente:

- Se ha dedicado bastante esfuerzo a probar todo el software: unidad, aceptación, carga, facilidad de uso, etcétera. La idea en el UP es llevar a cabo verificaciones de calidad y corrección tempranas, realistas y continuas, de manera que la pronta retroalimentación guíe a los desarrolladores a adaptar y mejorar el sistema, encontrando su “camino verdadero”.
- Los clientes se han involucrado regularmente en la evaluación del sistema parcial, para obtener retroalimentación con el objeto de adaptar y clarificar los requisitos. Y los clientes consiguen ver pronto un progreso visible del sistema.
- El sistema, a través de todos los subsistemas, se ha integrado completamente y estabilizado como versión base interna.

En aras de la brevedad, se han omitido muchas actividades del final de la Iteración 1 e inicio de la Iteración 2, puesto que esta presentación concede mucha importancia a introducir el A/DOO. A continuación presentamos comentarios sobre unas pocas de las muchas actividades que se han saltado:

- Al comienzo de la nueva iteración, utilice una herramienta CASE para llevar a cabo ingeniería inversa y generar los diagramas UML a partir del código fuente de la última iteración (los resultados forman parte del Modelo de Diseño del UP). Éstos se pueden imprimir a gran tamaño con un plotter y colgarse en las paredes de la sala del proyecto, lo que nos ayudaría a comunicar la representación del punto de partida del diseño lógico de la siguiente iteración.
- Se ha comenzado el análisis e ingeniería de usabilidad para la UI. Ésta es una técnica y una actividad extraordinariamente importantes para el éxito de muchos sistemas. Sin embargo, se trata de un tema extenso y no trivial, y queda fuera del alcance de este libro.
- Se ha comenzado el modelado e implementación de la base de datos.
- Se eligen los requisitos de la siguiente iteración cerca del final de la iteración anterior.
- Tiene lugar otro taller de requisitos de dos días (por ejemplo), en el que se escriben más casos de uso en formato completo. Durante la elaboración, mientras quizás se han diseñado e implementado el 10% de los requisitos de más riesgo, hay una actividad *paralela* para estudiar y definir en profundidad quizás el 80% de los casos de uso del sistema, aunque la mayoría de los requisitos no se implementarán hasta la construcción.
 - Entre los participantes se incluirán unos pocos desarrolladores (como el arquitecto de software) de la primera iteración, de manera que, durante este taller, la investigación y las preguntas se documentan a partir de la comprensión (y confusión) adquirida con la construcción rápida y real de parte del software. No hay nada como construir algo del software para descubrir qué es lo que realmente no

conocemos acerca de los requisitos —ésta es una idea clave en el UP y en el desarrollo iterativo—.

Simplificaciones en el caso de estudio

En un proyecto UP bien hecho, los requisitos que se eligen para las primeras iteraciones se organizan de acuerdo con el riesgo y el alto valor para el negocio, de manera que se identifican y se resuelven primero las cuestiones de más riesgo. Sin embargo, si este caso de estudio siguiera exactamente esta estrategia, no ayudaría a explicar las ideas y principios fundamentales del A/DOO en las primeras iteraciones. Por tanto, se han tomado algunas licencias con respecto a la prioridad de los requisitos, prefiriendo aquellos que apoyan los objetivos pedagógicos, en lugar de los objetivos de riesgo del proyecto.

21.3. Requisitos de la Iteración 2

La Iteración 2 de la aplicación del PDV NuevaEra maneja varios requisitos interesantes:

1. Soporte a las variaciones de servicios externos de terceras partes. Por ejemplo, se deben conectar al sistema diferentes calculadores de impuestos, cada uno con su propia interfaz. Del mismo modo con diferentes sistemas de contabilidad, etcétera. Cada uno ofrecerá un API y protocolo diferentes para un núcleo de las funciones comunes.
2. Reglas de fijación de precios complejas.
3. Reglas de negocio conectables.
4. Un diseño para actualizar una ventana del GUI cuando cambia el total de la venta.

Sólo se tendrán en cuenta estos requisitos (en esta iteración) en el contexto de los escenarios del caso de uso *Procesar Venta*.

Nótese que no se han descubierto requisitos nuevos; se identificaron durante la fase de inicio. Por ejemplo, el caso de uso *Procesar Venta* original señala el problema de la fijación de precios:

Escenario principal de éxito:

1. El Cliente llega a un terminal PDV con mercancías y/o servicios que comprar.
2. El Cajero le dice al sistema que cree una nueva venta.
3. El Cajero introduce el identificador del artículo.
4. El Sistema registra la línea de la venta y presenta la descripción del artículo, precio y suma parcial. ***El precio se calcula a partir de un conjunto de reglas de precios.***

Además, las secciones en la Especificación Complementaria registran los detalles de las reglas del dominio para los precios, y señalan la necesidad de soportar varios sistemas externos:

Especificación Complementaria			
...			
Interfaces			
Interfaces software			
Para la mayoría de los sistema externos de colaboración (calculador de impuestos, contabilidad, inventario,...) necesitamos ser capaces de conectar diversos sistemas y, por tanto, diversas interfaces.			
...			
Reglas de Dominio (Negocio)			
ID	Regla	Grado de variación	Fuente
REGLA4	Reglas de descuento al comprador. Ejemplos: Empleado—20% de descuento Clientes preferentes—10% de descuento Antiguos—15% de descuento	Alto. Cada tienda utiliza reglas diferentes.	Política de la tienda.
...	...	...	...
...			
Información en dominios de interés			
Fijación de precios			
Además de las reglas de fijación de precios que se describen en la sección de reglas del dominio, nótese que los artículos tienen un <i>precio original</i> , y opcionalmente, un <i>precio rebajado</i> . El precio de los artículos (antes de los descuentos adicionales) es el precio rebajado, si existe. Las organizaciones mantienen el precio original incluso si hay un precio rebajado, por razones de contabilidad e impuestos.			
...			

Desarrollo incremental para el mismo caso de uso a lo largo de la iteraciones

Debido a estos requisitos, en la Iteración 2 estamos volviendo sobre el caso de uso *Procesar Venta*, pero implementando más escenarios, de manera que el sistema crece incrementalmente. Es habitual trabajar en varios escenarios o características del mismo caso de uso a lo largo de varias iteraciones y extender el sistema gradualmente para finalmente manejar toda la funcionalidad requerida. Por otro lado, los casos de uso breves y simples, podrían implementarse completamente en una iteración.

La Iteración 1 hizo simplificaciones de manera que el problema y la solución no fueran excesivamente complejas de abordar. Una vez más —por la misma razón— se considera una cantidad relativamente pequeña de funcionalidad adicional.

En un proyecto de desarrollo real los requisitos que se han escogido en el libro para esta iteración no serían la elección indiscutible —otra posibilidad es actualizar el inventario—, gestionar los pagos a crédito, o un caso de uso completamente diferente. Sin embargo, ésta es una elección de suficiente riqueza con oportunidades de aprendizaje valiosas.

21.4. Refinamiento de los artefactos orientados al análisis en esta iteración

Modelo de casos de uso: casos de uso

Como resultado de los requisitos escogidos para esta iteración no se requiere ningún refinamiento de los casos de uso, aunque podrían cambiar como resultado de otras percepciones.

Sin embargo, además del diseño de objetos y la programación, en esta iteración tendrá lugar una actividad paralela de un breve taller de requisitos, con el que se investigarán y escribirán en detalle más casos de uso. Se revisarán los anteriores casos de uso en formato completo (por ejemplo, *Procesar Venta*) y probablemente se refinarán en base al conocimiento adquirido a partir de la Iteración 1. Algunas de estas actualizaciones de los casos de uso se podrían tener en cuenta en la siguiente iteración de la fase de elaboración, pero muchas se pospondrán hasta la construcción (porque no son significativas o arriesgadas para la arquitectura).

Modelo de casos de uso: DSS

Esta iteración abarca la inclusión del soporte para sistemas externos de terceras partes con varias interfaces, como un calculador de impuestos. El sistema del PDV NuevaEra se comunicará de manera remota con los sistemas externos. En consecuencia, deberían actualizarse los DSS para reflejar al menos algunas de las colaboraciones inter-sistemas, para clarificar cuáles son los nuevos eventos de nivel del sistema.

La Figura 21.1 ilustra un DSS para un escenario de pago a crédito, que requiere la colaboración con varios sistemas externos. Aunque en esta iteración no se maneja el pago a crédito, el diseñador (yo) ha dibujado un DSS basado en él (y probablemente varios otros también), para entender mejor la colaboración entre los sistemas y, por tanto, el soporte que se requiere para distintas interfaces en los sistemas externos.

Modelo del Dominio

Después de haber adquirido un poco de experiencia en el modelado del dominio, un modelador puede estimar si un conjunto de requisitos nuevos tendrán un mayor o menor impacto en el Modelo del Dominio, en términos de muchos nuevos conceptos, asociaciones y atributos. A diferencia de la iteración anterior, los requisitos que se van a abordar esta vez no comprenden muchos conceptos del dominio nuevos. Un breve estudio de los nuevos requisitos sugiere algo como *ReglaDePrecio* como concepto del dominio, pero probablemente no hay docenas de cosas nuevas.

En esta situación, es bastante razonable saltarse el refinamiento del Modelo del Dominio, y pasar rápidamente al trabajo de diseño, y dejar que tenga lugar el descubrimiento de nuevos conceptos del dominio durante el diseño de objetos, cuando los diseñadores están considerando detalladamente una solución. Una señal de madurez en el proceso con el UP es entender cuándo la creación de un nuevo artefacto añadirá un

valor significativo, o es una especie de paso mecánico “hacer trabajo” y es mejor saltarlo.

Esta flexibilidad es un arma de doble filo. Con mucha frecuencia, la flexibilidad de saltarse las actividades anteriores a la programación tiene lugar dentro de una creencia excesivamente optimista de que el problema se puede solucionar simplemente precipitándose hacia el código. Si verdaderamente puede, estupendo, porque la programación es el trabajo que realmente importa, no dibujar modelos del dominio. Por otro lado, la mayoría de los desarrolladores tienen historias donde un poco de reflexión, investigación y previsión antes de programar habrían reducido dolor y sufrimiento.

Modelo de caso de uso: contratos de las operaciones del sistema

En esta iteración no se van a considerar nuevas operaciones del sistema y, por tanto, no se necesitan los contratos. En cualquier caso, los contratos son únicamente una opción a tener en cuenta cuando la precisión detallada que ofrecen mejora las descripciones de los casos de uso.

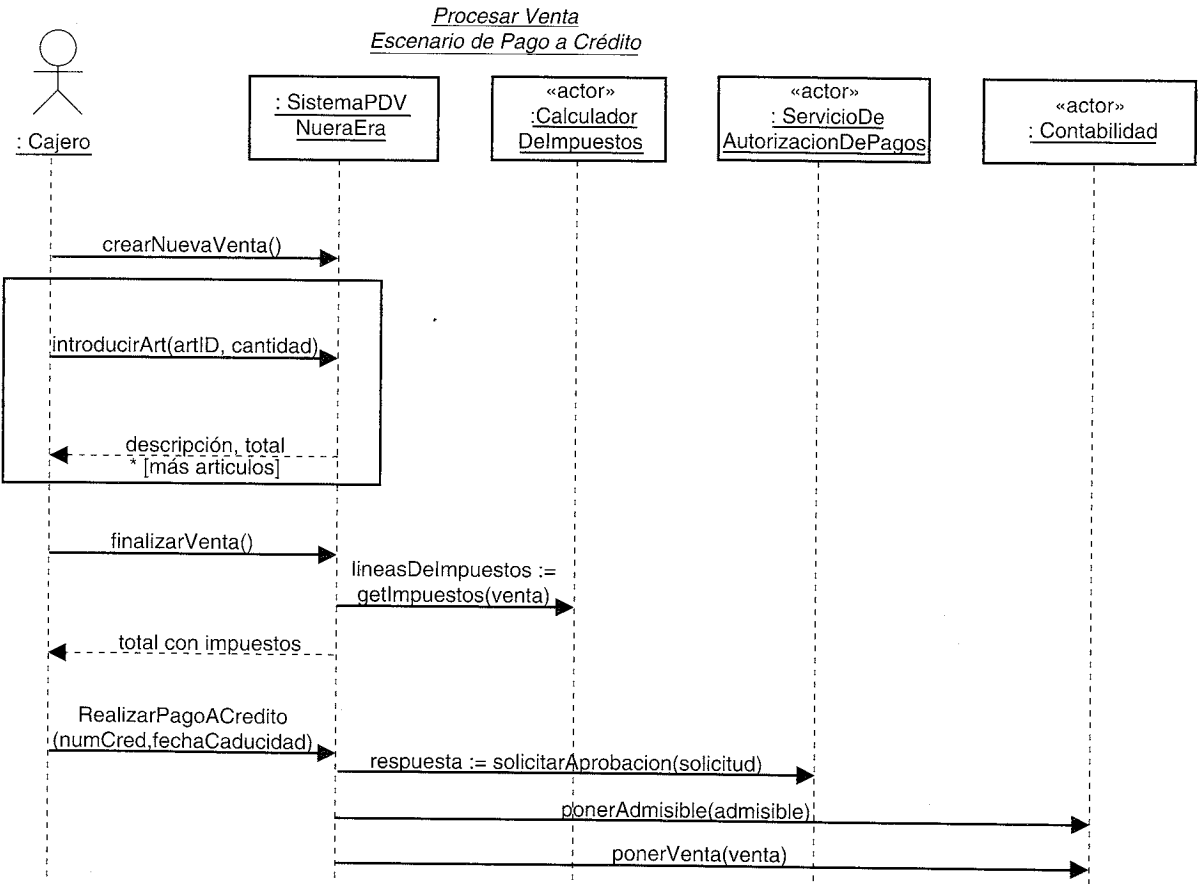


Figura 21.1. Un escenario DSS que ilustra algunos eventos externos.

GRASP: MÁS PATRONES PARA ASIGNAR RESPONSABILIDADES

La suerte es el residuo del diseño.
Branch Rickey

Objetivos

- Aprender a aplicar el resto de los patrones GRASP.

Introducción

- Anteriormente, estudiamos la aplicación de los cinco primeros patrones GRASP:
- Experto en Información, Creador, Alta Cohesión, Bajo Acoplamiento y Controlador.
- Los cuatro últimos patrones GRASP son:
- Polimorfismo.
 - Indirección.
 - Fabricación Pura.
 - Variaciones Protegidas.

Una vez que se hayan explicado, tendremos un vocabulario rico y de uso común con el que discutir los diseños. Y este vocabulario crecerá puesto que también se presentan (en capítulos siguientes) algunos de los patrones de diseño de la “pandilla de los cuatro” (GoF, *Gang-of-Four*) (como Estrategia y Factoría). Una frase corta como, “Sugiero una Estrategia generada a partir de una Factoría para soportar Variaciones Protegidas y Bajo Acoplamiento con respecto a <X>” revela mucha información sobre el diseño, puesto que el nombre de los patrones transmite de manera concisa un concepto de diseño complejo.

Este capítulo presenta el resto de los patrones GRASP, una ayuda para aprender los principios fundamentales según los cuales se asignan las responsabilidades a los objetos y se diseñan éstos.

Los capítulos siguientes introducen otros patrones útiles y los aplican al desarrollo de la segunda iteración de la aplicación del PDV NuevaEra.

22.1. Polimorfismo

Solución Cuando las alternativas o comportamientos relacionados varían según el tipo (clase), asigne la responsabilidad para el comportamiento —utilizando operaciones polimórficas— a los tipos para los que varía el comportamiento¹.

Corolario: No haga comprobaciones acerca del tipo de un objeto y no utilice la lógica condicional para llevar a cabo alternativas diferentes basadas en el tipo.

Problema ¿Cómo manejar las alternativas basadas en el tipo? ¿Cómo crear componentes software conectables (*pluggable*)?

Alternativas basadas en el tipo: La variación condicional es un tema fundamental en los programas. Si se diseña un programa utilizando sentencias de lógica condicional *if-then-else* o *case*, entonces, si aparece una nueva variación, requiere la modificación de la lógica de casos. Este enfoque dificulta que el programa se extienda con facilidad con nuevas variaciones porque se tiende a necesitar los cambios en varios sitios —en donde quiera que exista la lógica condicional.

Componentes software conectables: Viendo los componentes en las relaciones cliente-servidor, ¿cómo podemos sustituir un componente del servidor por otro, sin afectar al cliente?.

Ejemplo En la aplicación del PDV NuevaEra, se deben soportar diferentes sistemas externos de cálculo de impuestos de terceras partes (como Master-En-Impuestos e Impuestos-Pro); el sistema necesita ser capaz de integrarse con distintos de ellos. Cada calculador de impuestos tiene un interfaz diferente, con lo que hay un comportamiento parecido pero que varía para adaptarse a cada uno de estos interfaces externos o APIs. Un producto podría soportar un simple protocolo de sockets TCP, otro podría ofrecer un interfaz SOAP, y un tercero podría ofrecer un interfaz RMI de Java.

¿Qué objetos deberían ser los responsables de manejar estas interfaces diferentes de los calculadores de impuestos externos?

Puesto que el comportamiento para la adaptación del calculador varía según el tipo de calculador, según el Polimorfismo deberíamos asignar la responsabilidad de la adaptación a los diferentes objetos calculadores (o adaptadores de calculador), implementada con una operación polimórfica *getImpuestos* (ver Figura 22.1).

¹ El **polimorfismo** tiene varios significados relacionados. En este contexto, significa “asignar el mismo nombre a servicios en diferentes objetos” [Coad95] cuando los servicios son parecidos o están relacionados. Los diferentes tipos de objetos normalmente implementan un interfaz común o están relacionados en una jerarquía de implementación con una superclase común, pero esto depende del lenguaje; por ejemplo, los lenguajes con ligadura dinámica como Smalltalk no requieren esto.

Estos objetos adaptadores de los calculadores no son los calculadores externos, sino objetos software locales que representan los calculadores externos, o el adaptador para el calculador. Enviando un mensaje al objeto local, finalmente se hará una llamada al calculador externo a través de su API.

Cada método *getImpuestos* toma el objeto *Venta* como parámetro, de manera que el calculador pueda analizar la venta. La implementación de cada método *getImpuestos* será diferente: el *AdaptadorMasterEnImpuestos* adaptará la solicitud al API del Master-En-Impuestos, y así sucesivamente.

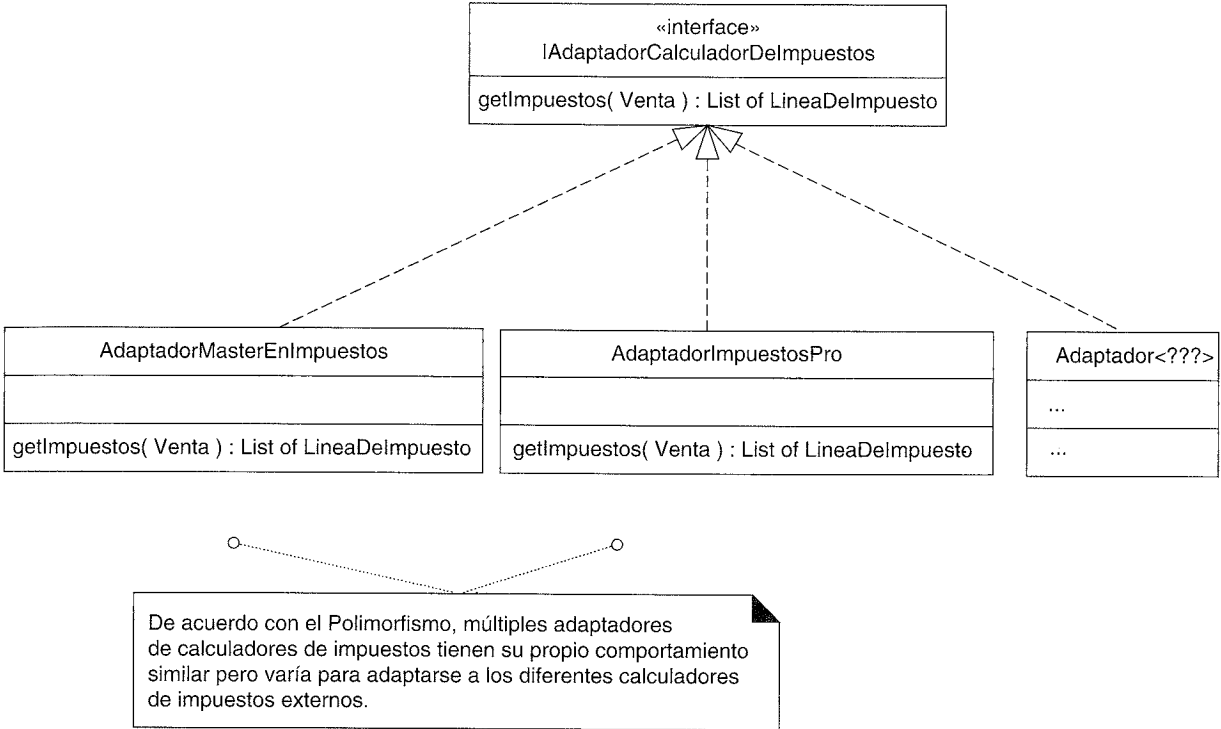


Figura 22.1. Polimorfismo en la adaptación a diferentes calculadores de impuestos externos.

Notación UML: La Figura 22.1 presenta alguna notación nueva para especificar las **interfaces** (un descriptor de operaciones sin implementación), la implementación de las interfaces, y los valores de retorno de tipo “colección”; la Figura 22.2 lo explica en detalle. Se utiliza un **estereotipo** UML para designar una interfaz; un estereotipo es un mecanismo para crear una nueva categoría de elementos. El nombre de un estereotipo se pone entre comillas francesas, como en «interface». Las comillas francesas son un signo ortográfico de un único carácter conocido sobre todo por su uso en la tipografía francesa para indicar una cita; pero citando a Rumbaugh, “quienes tengan problemas tipográficos pueden utilizar dobles corchetes angulares (<<>>)” [RJB99].

Discusión

El polimorfismo es un principio fundamental para diseñar cómo se organiza el sistema para gestionar variaciones similares. Según el Polimorfismo, un diseño basado en la asignación de responsabilidades puede extenderse fácilmente para manejar nuevas variaciones. Por ejemplo, la inclusión de una nueva clase adaptador de calculador con su propio método polimórfico *getImpuestos* tendrá un impacto menor en el diseño existente.

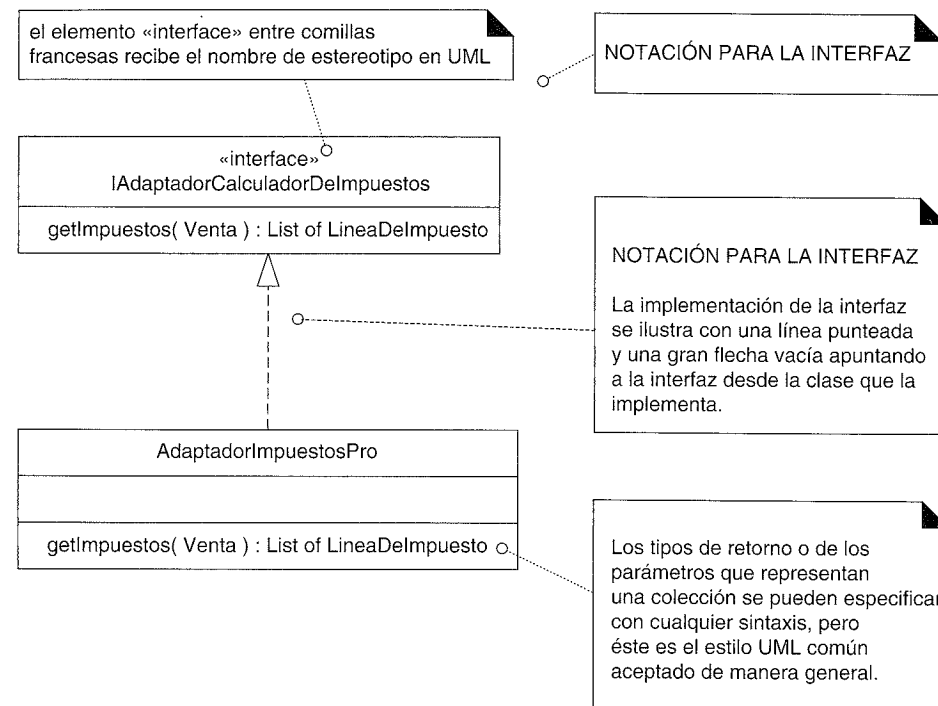


Figura 22.2. Notación UML para las interfaces y los tipos de retorno.

Contraindicaciones

Algunas veces, los desarrolladores diseñan sistemas con interfaces y polimorfismo para “futuras necesidades” especulativas frente a posibles variaciones desconocidas. Si el punto de variación está motivado sin ninguna duda por una variabilidad inmediata o muy probable, entonces, por supuesto, es razonable el esfuerzo de añadir la flexibilidad por medio del polimorfismo. Pero es conveniente una evaluación crítica, puesto que no es raro que se dedique un esfuerzo innecesario a preparar para el futuro un diseño con polimorfismo en puntos de variación que en realidad son improbables y nunca aparecerán realmente. Sea realista sobre la verdadera probabilidad de la posible variación antes de invertir en incrementar la flexibilidad.

Beneficios

- Se añaden fácilmente las extensiones necesarias para nuevas variaciones.
- Las nuevas implementaciones se pueden introducir sin afectar a los clientes.

Patrones Relacionados

- Variaciones Protegidas.
- Varios de los populares patrones de diseño GoF [GHJV95], que se estudiarán en este libro dependen del polimorfismo, entre los que se encuentran los patrones Adaptador, Command, Composite, Proxy, Estado y Estrategia.

También conocido como; parecido a

Elección del Mensaje, No preguntes “¿Qué clase?”.

22.2. Fabricación Pura

Solución

Asigne un conjunto de responsabilidades altamente cohesivo a una clase artificial o de conveniencia que no representa un concepto del dominio del problema —algo inventado para soportar alta cohesión, bajo acoplamiento y reutilización—.

Problema

Tal clase es una *fabricación* de la imaginación. Idealmente, las responsabilidades asignadas a esta fabricación soportan alta cohesión y bajo acoplamiento, de manera que el diseño de la fabricación es muy limpio, o *puro* —de ahí que sea una fabricación pura—.

Finalmente, una fabricación pura implica construir algo, ¡lo que hacemos cuando estamos desesperados!

¿Qué objetos deberían tener la responsabilidad cuando no quiere violar los objetivos de Alta Cohesión y Bajo Acoplamiento, u otros, pero las soluciones que ofrece el Experto (por ejemplo) no son adecuadas?

Los diseños orientados a objetos algunas veces se caracterizan por implementar como clases software las representaciones de los conceptos del dominio del mundo real para reducir el salto en la representación; por ejemplo, una clase *Venta* y *Cliente*. Sin embargo, hay muchas situaciones en las que la asignación de las responsabilidades sólo a las clases software de la capa del dominio da lugar a problemas en cuanto a cohesión y acoplamiento pobres, o que el potencial para reutilizar sea bajo.

Ejemplo

Por ejemplo, suponga que se necesita soporte para almacenar las instancias de *Venta* en una base de datos relacional. Según el Experto en Información, se puede justificar la asignación de esta responsabilidad a la propia clase *Venta*, puesto que la venta tiene los datos que se necesitan almacenar. Pero considere las siguientes implicaciones:

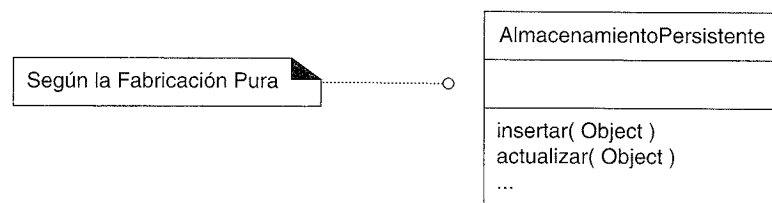
- La tarea requiere un número relativamente amplio de operaciones orientadas al soporte de la base de datos, ninguna relacionada con el concepto de ventas, luego la clase *Venta* deja de tener cohesión.
- La clase *Venta* tiene que acoplarse a la interfaz de la base de datos relacional (como JDBC en las tecnologías Java), luego el acoplamiento aumenta. E incluso el acoplamiento no se produce con otro objeto del dominio, sino con un tipo particular de interfaz de base de datos.
- Almacenar los objetos en una base de datos relacional es una tarea muy general que muchas clases necesitan soportar. La colocación de esta responsabilidad en la clase *Venta* da a entender que la reutilización va a ser pobre o que va a haber mucha duplicación en otras clases que hagan lo mismo.

Por tanto, aunque la *Venta* es una candidata lógica en virtud del Experto en Información para almacenarse a sí misma en una base de datos, nos lleva a un diseño con baja cohesión, alto acoplamiento y bajo potencial para reutilizar —exactamente el tipo de situación desesperada que pide que se cree algo—.

Una solución razonable es crear una nueva clase que es responsable únicamente de almacenar los objetos en algún tipo de medio de almacenamiento persistente, como una base de datos relacional; llamémosla *AlmacenamientoPersistente*². Esta clase es una *Fabricación Pura* —una invención de la imaginación—.

Obsérvese el nombre: *AlmacenamientoPersistente*. Es un concepto que se entiende, pero el nombre o concepto de “almacenamiento persistente” no es algo que uno encon-

² En un *framework* de persistencia real, se necesita finalmente más de una única clase de fabricación pura para crear un diseño razonable. Este objeto será una fachada *front-end* sobre un gran número de objetos de ayuda *back-end*.



traría en el Modelo del Dominio. Y si un diseñador le pregunta a una persona del negocio en una tienda, “¿Trabaja con objetos en almacenamiento persistente?” no le entendería. Ellos entienden conceptos como “venta” y “pago”. El *AlmacenamientoPersistente* no es un concepto del dominio, sino algo creado o fabricado para facilitar las cosas al desarrollador de software.

Esta Fabricación Pura soluciona los siguientes problemas de diseño:

- La *Venta* permanece bien diseñada, con alta cohesión y bajo acoplamiento.
- La clase *AlmacenamientoPersistente* es relativamente cohesiva, teniendo como único objetivo el almacenamiento o inserción de los objetos en un medio de almacenamiento persistente.
- La clase *AlmacenamientoPersistente* es un objeto muy genérico y reutilizable.

La creación de una fabricación pura en este ejemplo es exactamente la situación en la que se requiere su uso —sustituir un mal diseño basado en el Experto, con cohesión y acoplamiento pobres, por un buen diseño en el que el potencial para reutilizar es mayor—.

Observe, como con todos los patrones GRASP, se concede mucha importancia al lugar donde se debería colocar la responsabilidad. En este ejemplo las responsabilidades se cambian de la clase *Venta* (motivada según el Experto) a una Fabricación Pura.

Discusión

El diseño de los objetos se puede dividir en general en dos grupos:

1. Los escogidos de acuerdo a una **descomposición de la representación**.
2. Los escogidos según una **descomposición del comportamiento**.

Por ejemplo, se crean las clases software como la *Venta* de acuerdo a una descomposición de la representación; la clase software está relacionada o representa una cosa en un dominio. La descomposición de la representación es una estrategia común en el diseño de objetos y favorece el objetivo de salto en la representación reducido. Pero algunas veces, deseamos asignar responsabilidades agrupando comportamientos o algoritmos, sin que interese crear una clase con un nombre u objetivo que esté relacionado con un concepto del dominio del mundo real.

Un buen ejemplo es un objeto “algoritmo” como un *GeneradorDeTablaDeContenidos*, cuyo objetivo es (sorpresa) generar una tabla de contenidos y lo creó un desarrollador como clase de ayuda o de conveniencia, sin preocuparse por elegir un nombre del vocabulario del dominio de libros y documentos. Existe como clase de conveniencia concebida por el desarrollador para agrupar algún comportamiento o métodos relacionados y, por tanto, está motivada según la *descomposición de comportamiento*.

En cambio, una clase software denominada *TablaDeContenidos* está inspirada siguiendo una *descomposición de la representación*, y debería contener información consistente con nuestra concepción del dominio real (como los nombres de los capítulos).

No es crítico identificar una clase como Fabricación Pura. Es un objetivo educativo transmitir la idea general de que algunas clases software se inspiran de acuerdo con las representaciones del dominio, y otras simplemente se “crean” por conveniencia del diseñador de objetos. Estas clases de conveniencia normalmente se crean para agrupar algún comportamiento común y, por tanto, se inspiran según una descomposición de comportamiento en lugar de una descomposición de la representación.

Dicho de otro modo, una Fabricación Pura normalmente se organiza en base a funcionalidad relacionada, y de este modo es un objeto centrado en la función o de comportamiento.

Muchos de los patrones de diseño orientados a objetos existentes son ejemplo de Fabricación Pura: Adaptador, Estrategia, *Command*, etcétera [GHJV95].

Como comentario final merece la pena reiterar: Algunas veces la solución que ofrece el Experto en Información no es conveniente. Aunque el objeto sea un candidato porque tiene mucha de la información relacionada con la responsabilidad, en otros aspectos, su elección nos lleva a un diseño pobre, normalmente debido a problemas en la cohesión o el acoplamiento.

Beneficios

- Se soporta Alta Cohesión puesto que las responsabilidades se factorizan en una clase de grano fino que sólo se centra en un conjunto muy específico de tareas relacionadas.
- El potencial para reutilizar podría aumentar debido a la presencia de clases de Fabricación Pura de grano fino cuyas responsabilidades tienen aplicación en otras aplicaciones.

Contraindicaciones

Algunas veces, los inexpertos en el diseño de objetos y que conocen bien la descomposición u organización del software en base a sus funciones, abusan de la descomposición de comportamiento en los objetos de Fabricación Pura. Exagerando, sucede que las funciones se convierten en objetos. No existe nada inherentemente incorrecto en la creación de objetos “función” o “algoritmo”, pero necesita equilibrarse con la capacidad de diseñar con descomposición de la representación, como la capacidad de aplicar el Experto en Información de manera que una clase de la representación como la *Venta* también tenga responsabilidades. El Experto en Información enuncia el objetivo de colocar las responsabilidades con los objetos que conocen la información necesaria para esas responsabilidades, lo cual tiende a favorecer la disminución del acoplamiento. Si se abusa, la Fabricación Pura podría dar lugar a demasiados objetos de comportamiento con responsabilidades que *no* se colocaron con la información necesaria para su realización, lo que puede afectar negativamente al acoplamiento. El síntoma habitual es que la mayoría de los datos contenidos en los objetos se pasan a otros objetos para trabajar con ellos.

Patrones y Principios Relacionados

- Bajo Acoplamiento.
- Alta Cohesión.
- Usualmente una Fabricación Pura asume responsabilidades de las clases del dominio a las que se les asignaría esas responsabilidades en base al patrón Experto.
- Todos los patrones de diseño GoF [GHJV95], como el Adaptador, *Command*, Estrategia, etcétera, son Fabricaciones Puras.
- Prácticamente el resto de patrones de diseño son Fabricaciones Puras.

22.3. Indirección

Solución Asigne la responsabilidad a un objeto intermedio que medie entre otros componentes o servicios de manera que no se acoplen directamente.

El intermediario crea una *indirección* entre los otros componentes.

Problema ¿Dónde asignar una responsabilidad, para evitar el acoplamiento directo entre dos (o más) cosas? ¿Cómo desacoplar los objetos de manera que se soporte el bajo acoplamiento y el potencial para reutilizar permanezca más alto?

Ejemplos **AdaptadorCalculadorDeImpuestos**

Estos objetos actúan como intermediarios con los calculadores de impuestos externos. Mediante el polimorfismo, proporcionan una interfaz consistente para los objetos internos y ocultan las variaciones en las APIs externas. Añadiendo un nivel de indirección y el polimorfismo, los objetos adaptador protegen el diseño interno frente a las variaciones de las interfaces externas (ver Figura 22.3).

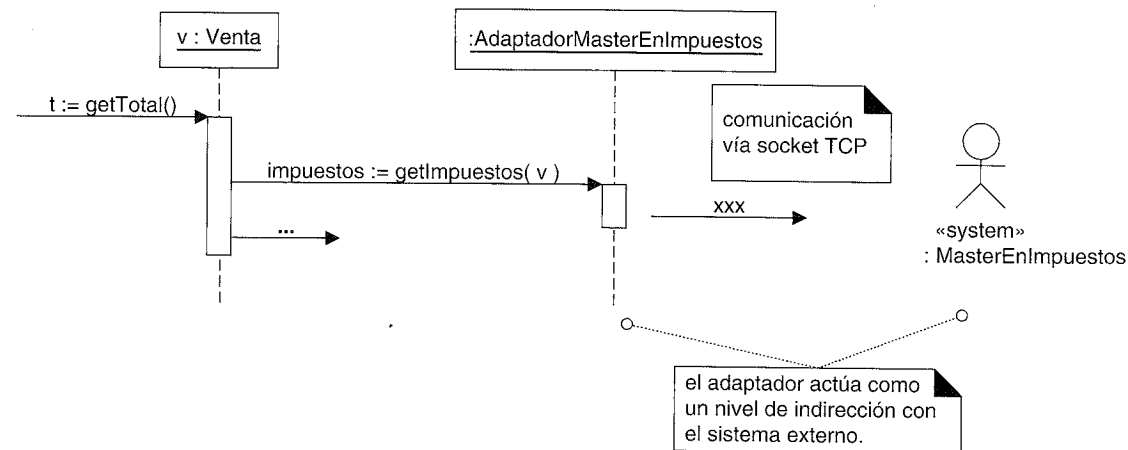


Figura 22.3. Indirección por medio del adaptador.

AlmacenamientoPersistente

El ejemplo de Fabricación Pura de desacoplar la *Venta* de los servicios de base de datos relacional mediante la incorporación de una clase *AlmacenamientoPersistente*, también es un ejemplo de asignación de responsabilidades para soportar la Indirección. El *AlmacenamientoPersistente* actúa como intermediario entre la *Venta* y la base de datos.

Discusión “La mayoría de los problemas en informática se pueden resolver mediante otro nivel de indirección” es un viejo dicho de particular relevancia en los diseños orientados a objetos³.

³ ¿Si algún dicho es viejo en informática! He olvidado la fuente (¿Parnas?). Nótese que también existe el dicho contrario: “¡La mayoría de los problemas en ejecución se pueden resolver eliminando alguna capa de indirección!”.

Del mismo modo que muchos de los patrones de diseño existentes son especializaciones de Fabricación Pura, muchos son también especializaciones de Indirección. Algunos ejemplos son Adaptador, Fachada y Observador. Además, muchas Fabricaciones Puras se generan debido a la Indirección. El motivo para la Indirección normalmente es el Bajo Acoplamiento; se añade un intermediario para desacoplar otros componentes o servicios.

Beneficios

- Disminuir el acoplamiento entre los componentes.

Patrones y Principios Relacionados

- Variaciones Protegidas.
- Bajo Acoplamiento.
- Muchos patrones GoF como Adaptador, Puente, Fachada, Observador y Mediador [GHJV95].
- Muchos intermediarios de Indirección son Fabricaciones Puras.

22.4. Variaciones Protegidas

Solución

Identifique los puntos de variaciones previstas o de inestabilidad; asigne responsabilidades para crear una interfaz estable alrededor de ellos.

Nota: El término “interfaz” se utiliza en el sentido más general de una vista de acceso; no sólo significa literalmente algo como una interfaz de Java o COM.

Problema

¿Cómo diseñar objetos, subsistemas y sistemas de manera que las variaciones o inestabilidades en estos elementos no tengan un impacto no deseable en otros elementos?

Ejemplo

Por ejemplo, el problema anterior del calculador de impuestos externo y su solución con el Polimorfismo ilustra las Variaciones Protegidas (Figura 22.1). El punto de inestabilidad o variación lo forman las diferentes interfaces o APIs de los calculadores de impuestos externos. El sistema de PDV necesita ser capaz de integrarse con muchos sistemas de cálculo de impuestos existentes, y también con futuros calculadores de terceras partes que no existen todavía.

Añadiendo un nivel de indirección, una interfaz, y utilizando el polimorfismo con varias implementaciones de *IAdaptadorCalculadorDeImpuestos*, se consigue proteger al sistema de las variaciones en las APIs externas. Los objetos internos colaboran con una interfaz estable; las distintas implementaciones del adaptador ocultan las variaciones de los sistemas externos.

Discusión

Cockburn publicó por primera vez las Variaciones Protegidas (VP) como patrón en [VCK96], aunque este principio de diseño tan fundamental ha estado por ahí durante décadas bajo distintos términos.

Mecanismos motivados por VP

VP es un principio fundamental que motiva la mayoría de los mecanismos y patrones en la programación y el diseño destinados a proporcionar flexibilidad y protección frente a las variaciones.

A un nivel, se puede ver la madurez de un desarrollador o arquitecto en su conocimiento creciente de mecanismos de amplia aplicación para conseguir VP, elegir las batallas VP apropiada que merecen la pena pelear, y su capacidad para escoger una solución de VP adecuada. En las primeras etapas, uno aprende acerca de la encapsulación de datos, interfaces y polimorfismo —todos ellos mecanismos básicos para alcanzar VP—. Más tarde, uno aprende técnicas como los lenguajes basados en reglas, intérpretes de reglas, diseños basados en reflexión y metadatos, máquinas virtuales, etcétera —todas ellas se pueden aplicar para proteger contra alguna variación—.

Por ejemplo:

Mecanismos básicos de Variaciones Protegidas

VP motiva la encapsulación de datos, interfaces, polimorfismo, indirección y estándares. Nótese que los componentes como gestores (*brokers*) y máquinas virtuales son ejemplos complejos de indirección para conseguir VP.

Diseños dirigidos por los datos

Los diseños dirigidos por los datos cubren una amplia familia de técnicas entre las que se encuentran la lectura de códigos, valores, rutas de ficheros de clase, nombres de clase, etcétera, procedentes de una fuente externa para cambiar el comportamiento, o “parametrizar” un sistema de alguna manera en tiempo de ejecución. Otras variantes pueden ser hojas de estilo, metadatos para la correspondencia objeto-relacional, ficheros de propiedades, lectura de distribución de los elementos de una ventana y muchos más. Se protege al sistema del impacto de variaciones de los datos, metadatos o declarativas registrando externamente la variante, leyéndola, y razonando sobre ella.

Búsqueda de servicios

La búsqueda de servicios incluye técnicas como la utilización de servicios de nombres (por ejemplo, JNDI de Java) o intermediarios (*traders*) para obtener un servicio (por ejemplo, Jini de Java, o UDDI para los servicios Web). Se protegen a los clientes de las variaciones en la ubicación de los servicios, utilizando la interfaz estable de la búsqueda del servicio. Es un caso especial de diseño dirigido por los datos.

Diseños dirigidos por un intérprete

Los diseños dirigidos por un intérprete comprenden los intérpretes de reglas que ejecutan las reglas leídas de una fuente externa, intérpretes de script o lenguajes que leen y ejecutan programas, máquinas virtuales, motores de redes neuronales que ejecutan redes, motores lógicos de restricciones que leen y razonan con conjuntos de restricciones, etcétera. Este enfoque permite cambiar o parametrizar el comportamiento de un sistema por medio de expresiones lógicas externas. Se protege al sistema del impacto de las variaciones lógicas registrando externamente la lógica, leyéndola, y utilizando un intérprete.

Diseños reflexivos o de meta-nivel

Un ejemplo de este enfoque es utilizar el *java.beans.Introspector* para obtener un objeto *BeanInfo*, solicitar el objeto *Method* para el método de acceso (*getter*) para la propiedad del *bean* X, e invocar *Method.invoke*. Se protege al sistema del impacto de la lógica o variaciones externas del código mediante algoritmos reflexivos que utilizan introspección y servicios de metalenguaje. Podría considerarse un caso especial de diseño dirigido por los datos.

Acceso uniforme

Algunos lenguajes como Ada, Eiffel y C# soportan un constructor sintáctico de manera que se expresan de la misma manera el acceso a un método y a un campo. Por ejemplo, *unCirculo.radio* podría invocar al método *radio():float* o referirse directamente al campo público, dependiendo de la definición de la clase. Podemos cambiar de campos públicos a métodos de acceso, sin cambiar el código del cliente.

Principio de Sustitución de Liskov (PSL)

PSL [Liskov88] formaliza el principio de protección contra las variaciones en implementaciones diferentes de una interfaz, o una subclase que extiende una superclase.

Citando textualmente:

Lo que se quiere aquí es algo como la siguiente propiedad de sustitución: si para cada objeto *o1* de tipo *S* hay un objeto *o2* de tipo *T* tal que para todos los programas *P* definidos en términos de *T*, el comportamiento de *P* no varía cuando se sustituye *o1* por *o2* entonces *S* es un subtipo de *T* [Liskov88].

Informalmente, el software (métodos, clases,...) que hace referencia a un tipo *T* (alguna interfaz o superclase abstracta) debería trabajar correctamente o como se espera con cualquier implementación o subclase de *T* que la sustituya —llamémosla *S*—. Por ejemplo:

```
public void añadirImpuestos
    (IAdaptadorCalculadorDeImpuestos calculador, Venta venta)
{
    List lineasDeImpuestos = calculador.getImpuestos( venta );
    //...
}
```

Para este método *añadirImpuestos*, no importa la implementación de *IAdaptadorCalculadorDeImpuestos* que se pase como argumento real, el método debería continuar funcionando “como se espera”. PSL es una idea simple, intuitiva para la mayoría de los desarrolladores de software, que formaliza esta intuición.

Diseños de ocultación de la estructura

En la primera edición de este libro, se enunció un importante principio de diseño de objetos denominado **No Hable con Extraños** o la **Ley de Demeter** [Lieberherr88] como uno de los nueve patrones GRASP. Brevemente, significa evitar crear diseños que recorren largos caminos de la estructura de objetos y envía mensajes (o habla) con objetos distantes, indirectos (extraños). Tales diseños son frágiles con respecto a los cambios en las estructuras de los objetos —un punto frecuente de inestabilidad—. Pero en la segunda edición el principio más general VP sustituye a No Hable con Extraños, porque el segundo es un caso especial del primero. Es decir, un mecanismo para lograr proteger de los cambios en la estructura es aplicar las reglas de No Hable con Extraños.

No Hable con Extraños restringe los objetos a los que deberías enviar los mensajes dentro de un método. Establece que en un método, sólo deberían enviarse mensajes a los siguientes objetos:

1. El objeto *this* (o *self*).
2. Un parámetro del método.
3. Un atributo de *this*.
4. Un elemento de una colección que es un atributo de *this*.
5. Un objeto creado en el método.

La intención es evitar el acoplamiento de un cliente con conocimiento de objetos indirectos y las conexiones entre objetos.

Los objetos directos son “conocidos” del cliente, los objetos indirectos son “extraños”. Un cliente debería hablar con conocidos y evitar hablar con extraños.

A continuación presentamos un ejemplo que viola (ligeramente) el principio No Hables con Extraños. Los comentarios explican la violación.

```
class Registro
{
    private Venta venta;

    public void metodoAlgoFragil()
    {
        //venta.getPago() envía un mensaje a un "conocido" (según #3)

        //pero en venta.getPago().getCantidadEntregada()
        //el mensaje getCantidadEntregada() se aplica a un Pago "extraño"

        Dinero cantidad = venta.getPago().getCantidadEntregada();

        //...
    }
    //...
}
```

Este código recorre conexiones estructurales a partir de un objeto conocido (la *Venta*) a un objeto extraño (el *Pago*), y entonces le envía el mensaje. Es ligeramente frágil, ya que depende del hecho de que los objetos *Venta* se conecten a los objetos *Pago*. En realidad, no es probable que esto sea un problema.

Pero, considere el siguiente fragmento, que llega más lejos a lo largo del camino de la estructura:

```
public void metodoMasFragil()
{
    TitularCuenta titular =
        venta.getPago().getCuenta().getTitularCuenta();

    //...
}
```

El ejemplo es artificial, pero puede ilustrar el patrón: llegar más lejos a lo largo de un camino de conexiones de objetos para enviar un mensaje a un objeto distante e indirecto —hablando con un extraño alejado—. El diseño está acoplado a una estructura particular de cómo están conectados los objetos. Cuanto más lejos a lo largo de un camino recorre el programa, más frágil es.

Karl Lieberherr y sus colegas han investigado sobre buenos principios de diseño de objetos, en el marco del proyecto Demeter. Esta Ley de Demeter (No Hable con Extraños) se identificó debido a que vieron que con frecuencia se producían cambios e inestabilidad en la estructura de los objetos, y como resultado frecuentes roturas en el código que estaba acoplado con el conocimiento de las conexiones de los objetos.

Pero, como examinaremos en la siguiente sección “VP especulativo y escoja sus batallas”, no siempre es necesario proteger contra esto; depende de la inestabilidad de la estructura de los objetos. En las librerías estándar (como las librerías de Java) las conexiones estructurales entre las clases de los objetos son relativamente estables. En sistemas maduros, la estructura es más estable. En sistemas nuevos en las primeras iteraciones, no es estable.

En general, cuanto más lejos se recorre a lo largo de un camino, más frágil es y, por tanto, es más útil ser conforme con el principio No Hable con Extraños.

Si obedecemos estrictamente esta ley —protección contra las variaciones estructurales— es necesario que se añadan nuevas operaciones públicas a los “conocidos” de un objeto; estas operaciones proporcionan en último término la información deseada, y ocultan cómo se obtuvo. Por ejemplo, para soportar No Hable con Extraños en los dos casos anteriores:

```
//caso 1
Dinero cantidad = venta.getCantidadEntregadaEnPago();

//caso 2
TitularCuenta titular = venta.getTitularCuentaDelPago();
```


Contraindicaciones**Advertencia: VP especulativo y escoja sus batallas**

En primer lugar, merece la pena definir dos puntos de cambio:

- **Punto de variación:** variaciones en el sistema actual, existente o en los requisitos, como las múltiples interfaces de los calculadores de impuestos que se deben soportar.
- **Puntos de evolución:** puntos especulativos de variación que podrían aparecer en el futuro, pero que no están presentes en los requisitos actuales⁴.

VP se aplica tanto a los puntos de variación como de evolución

Una advertencia: algunas veces el coste de “futuras necesidades” especulativas en los puntos de evolución pesa más que el coste en el que se incurre por un diseño simple, más “frágil” que se revisa cuando sea necesario en respuesta a las verdaderas presiones de cambio. Es decir, el coste de diseñar la protección en los puntos de evolución puede ser más alto que rehacer un diseño simple.

Por ejemplo, recuerdo un sistema de manejo de mensajes del busca en el que el arquitecto incluyó un lenguaje de script y un intérprete para dar soporte a la flexibilidad y variaciones protegidas en un punto de evolución. Sin embargo, durante la revisión en una versión incremental, se eliminó el complejo (e ineficiente) script —simplemente no era necesario—. Y cuando empecé con la programación orientada a objetos (al principio de los ochenta) padecí la enfermedad de “generalizitis” en la que solía pasar muchas horas creando superclases de las clases que realmente necesitaba escribir. Yo haría todo muy general y flexible (y protegido contra las variaciones), para esas situaciones futuras en las que realmente sería beneficioso —lo que no sucedía nunca—. Era un pobre juez de cuando merecía la pena el esfuerzo.

La idea no es abogar por diseños frágiles que hay que rehacer. Si la necesidad de flexibilidad y protección de cambios es realista, entonces está motivada la aplicación de VP. Pero si es para futuras necesidades o supuesta “reutilización” con probabilidades muy inciertas, entonces se requiere un modo de ver las cosas restringido y crítico.

Los desarrolladores sin experiencia tienden a realizar diseños frágiles, los desarrolladores de nivel intermedio tienden a unos diseños excesivamente flexibles y generalizados (de unas formas que nunca se utilizan). Los diseñadores expertos eligen con perspicacia; quizás un diseño simple y frágil en el que existe un equilibrio entre el coste del cambio y su probabilidad.

- Se añaden fácilmente las extensiones que se necesitan para nuevas variaciones.
- Se pueden introducir nuevas implementaciones sin afectar a los clientes.
- Se reduce el acoplamiento.
- Puede disminuirse el impacto o coste de los cambios.

Beneficios

⁴ En el UP, los puntos de evolución se pueden documentar formalmente en **Casos de Cambio**; cada uno describe aspectos relevantes de un punto de evolución a beneficio de un futuro arquitecto.

Patrones y Principios Relacionados

- La mayoría de los patrones y principios de diseño son mecanismos para variaciones protegidas, entre los que se encuentran el polimorfismo, las interfaces, indirectión, encapsulación de datos, la mayoría de los patrones de diseño GoF, etcétera.
- En [Pree95] los puntos de variación y evolución reciben el nombre de “puntos calientes”.

También conocido como; Parecido a

VP es esencialmente lo mismo que los principios de ocultación de la información y de abierto-cerrado, que son términos más antiguos. Como patrón “oficial” en la comunidad de los patrones, fue Cockburn en [VCK96] quien lo denominó “Variaciones Protegidas” en 1996.

Ocultación de la Información

El famoso artículo de David Parnas *On the Criteria To Be Used in Decomposing Systems Into Modules* (Criterios a Utilizar para Descomponer los Sistemas en Módulos) [Parnas72] es un ejemplo de los clásicos que siempre se citan pero que rara vez se leen. En él, Parnas introduce el concepto de **ocultación de información**. Quizás debido a que el término suena como la idea de encapsulación de datos, se ha mal interpretado como eso, y algunos libros definen erróneamente los conceptos como sinónimos. Más bien, lo que Parnas quiso decir con la ocultación de información era ocultar información sobre los diseños a otros módulos, en los puntos de cambio difíciles o probables. Citando literalmente su explicación sobre la ocultación de información como principio de diseño conductor:

Proponemos en lugar de eso que uno comience con una lista de decisiones de diseño difíciles o decisiones de diseño que es probable que cambien. Cada módulo entonces se diseña para ocultar tal decisión a los otros.

Esto es, el principio de Parnas de ocultación de información es el mismo que se enuncia en VP, y no es simplemente encapsulación de datos —que no es más que una de las muchas técnicas para ocultar información acerca del diseño—. Sin embargo, el término ha sido tan ampliamente reinterpretado como sinónimo de encapsulación de datos que ya no es posible utilizarlo en el sentido original sin malentendidos.

Principio Abierto-Cerrado

El **Principio Abierto-Cerrado** (PAC), descrito por Bertrand Meyer en [Meyer88] es esencialmente equivalente al patrón VP y a la ocultación de información. Una definición del PAC es:

Los módulos deberían ser tanto abiertos (para extenderse; adaptables) como cerrados (el módulo está cerrado a las modificaciones que afecten a los clientes).

El PAC y las VP son esencialmente dos expresiones del mismo principio, que resaltan diferentes puntos: protección en los puntos de variación y evolución. En el PAC, los “módulos” comprenden todos los elementos de software discretos, como los métodos, clases, subsistemas, aplicaciones, etcétera.

En el contexto del PAC, la frase “cerrado con respecto a X” significa que los clientes no se ven afectados si X cambia. Por ejemplo, “la clase está cerrada con respecto a las definiciones de los campos de instancia” mediante el mecanismo de encapsulación de datos con campos privados y métodos de acceso públicos. Al mismo tiempo, están abiertas a la modificación de las definiciones de los datos privados, porque los clientes externos no están acoplados directamente a los datos privados.

Otro ejemplo podría ser, “los adaptadores de los calculadores de impuestos están cerrados con respecto a su interfaz pública” mediante la implementación de la interfaz estable *IAdaptadorCalculadorDeImpuestos*. Sin embargo, los adaptadores están abiertos a la extensión siendo modificados de manera privada en respuesta a los cambios en las APIs de los calculadores de impuestos externos, de manera que no afectan a sus clientes.

DISEÑO DE LAS REALIZACIONES DE CASOS DE USO CON LOS PATRONES DE DISEÑO GoF

Cualquier cosa que tú puedas hacer, yo puedo hacerla meta.
Daniel Dennett

Objetivos

- Aplicar los patrones de diseño GRASP y GoF al diseño del caso de estudio NuevaEra.

Introducción

Este capítulo estudia el diseño de objetos para las realizaciones de los casos de usos de la siguiente iteración del caso de estudio NuevaEra, que aborda el soporte para el acceso a los servicios externos de terceras partes cuyas interfaces podrían variar, de reglas más complejas para establecer el precio de los productos y de reglas del negocio conectables.

En el contexto de los problemas de diseño que se discutirán, también se introducirá nueva notación UML muy utilizada.

Lo importante es mostrar cómo aplicar los patrones GoF y los GRASP más básicos. Se pretende ilustrar que el diseño de objetos y la asignación de responsabilidades se puede explicar y aprender en base a la aplicación de los patrones —un vocabulario de principios y estilos que se pueden combinar para diseñar los objetos—.

Los patrones de la “pandilla de los cuatro” (Gang-of-Four)

Los patrones adicionales que se presentan en este capítulo proceden de *Design Patterns* [GHJV95], un libro básico y muy popular que presenta 23 patrones que son útiles durante el diseño de objetos. Puesto que el libro fue escrito por cuatro autores, estos patrones se conocen como los patrones de la “pandilla de los cuatro” o patrones “GoF”¹.

Este capítulo proporciona una introducción a algunos de los patrones GoF más utilizados; capítulos posteriores presentarán más². Se recomienda un estudio completo y cuidadoso del libro *Design Patterns* para iniciarse como diseñador de objetos, aunque el libro asume que el lector ya es un diseñador con alguna experiencia; este libro ofrece una introducción.

Un vocabulario compartido

Además del vocabulario visual de la notación UML, al final de este capítulo tendremos un vocabulario de diseño compartido más rico, en términos de nombres de patrones. Por tanto, será posible comunicar progresivamente ideas de diseño del software sobre todo mediante diagramas UML, con algunas notas conectadas que indican los patrones (Indirección, Estrategia,...) que se van a aplicar.

23.1. Adaptador (GoF)

El problema que se presentó en el capítulo anterior para motivar el patrón Polimorfismo, y su solución, es más concretamente un ejemplo del patrón GoF **Adaptador**.

Adaptador (<i>Adapter</i>)
<i>Contexto/Problema</i>
¿Cómo resolver interfaces incompatibles, o proporcionar una interfaz estable para componentes parecidos con diferentes interfaces?
<i>Solución</i>
Convierta la interfaz original de un componente en otra interfaz, mediante un objeto adaptador intermedio.

Recordemos que: el sistema de PDV NuevaEra necesita soportar diferentes tipos de servicios externos de terceras partes, entre los que se encuentran los calculadores de impuestos, servicios de autorización de pagos, sistemas de inventario, y sistemas de contabilidad, entre otros. Cada uno tiene un API diferente, que no puede ser cambiada.

Una solución es añadir un nivel de indirección con objetos que adapten las distintas interfaces externas a una interfaz consistente que es el que se utiliza en la aplicación. La solución se ilustra en la Figura 23.1.

¹ Con una referencia tangencial a la política China.

² En la práctica, quizás unos 15 de estos 23 patrones son los que se utilizan con frecuencia.

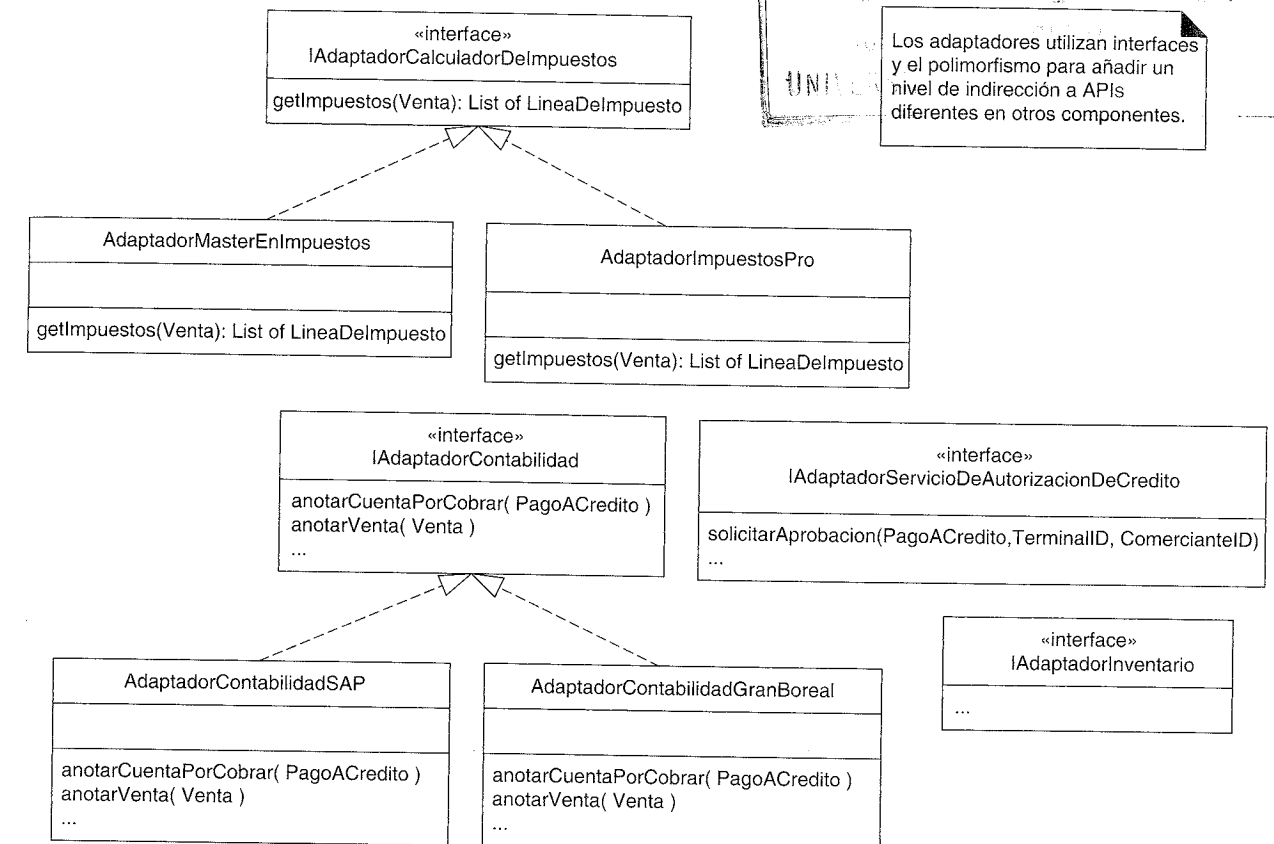


Figura 23.1. El patrón Adaptador.

Como se ilustra en la Figura 23.2, una instancia de un adaptador particular se instanciará para el servicio externo elegido³, como SAP para contabilidad, y adaptará la solicitud *anotarVenta* a la interfaz externa, tal como una interfaz SOAP XML sobre HTTPS para un servicio Web de intranet ofrecido por SAP.

Notación UML: Obsérvese en la Figura 23.2 el uso de un “piruli” de interfaz para indicar que la instancia *AdaptadorContabilidadSAP* implementa un interfaz relevante.

Polimorfismo, Indirección y Variaciones Protegidas (GRASP)

La aplicación anterior del patrón Adaptador es una especialización de los componentes básicos GRASP. Ofrece Variaciones Protegidas de los cambios en las interfaces externas o paquetes de terceras partes mediante el uso de un objeto de Indirección que aplica interfaces y Polimorfismo.

³ En la Arquitectura de Conexión de J2EE, estos adaptadores a los servicios externos se denominan de manera más específica **adaptadores de recursos**.

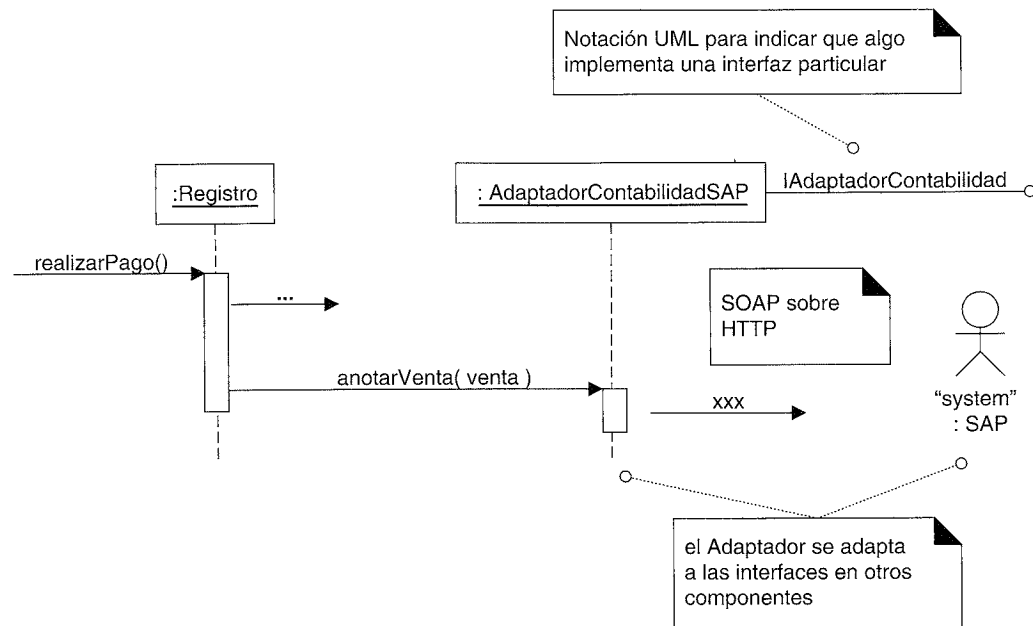


Figura 23.2. Utilización de un adaptador.

Nótese que se pueden analizar patrones mucho más complejos y especializados en función de la familia GRASP básica. Existen cientos de patrones de diseño publicados, y aunque es útil estudiarlos para acelerar el aprendizaje, entender sus conceptos básicos subyacentes (Variaciones Protegidas, Bajo Acoplamiento, Polimorfismo, Indirección...) nos ayudan a abrirnos camino a través de los innumerables detalles y ver el “alfabeto” esencial de las técnicas de diseño que se van a aplicar.

Convención de nombres: ¿incluimos el nombre del patrón en el nombre del tipo?

Nótese que los nombres de los tipos incluyen el nombre del patrón “Adaptador”. Éste es un estilo relativamente común y tiene la ventaja de poder comunicar fácilmente a los que leen el código o los diagramas el patrón de diseño que se va a utilizar.

23.2. Descubrimientos del “análisis” durante el diseño: Modelo del Dominio

Observe que en el diseño del Adaptador en la Figura 23.1, la operación *getImpuestos* devuelve una lista de *LineaDeImpuesto*. Es decir, al reflexionar e investigar en profundidad el modo de gestionar los impuestos y cómo funcionan los calculadores de impuestos, el diseñador (yo) se dio cuenta de que se asocia una lista de líneas de impuestos con una venta, como impuesto del gobierno central, impuesto del gobierno regional, etcétera (¡siempre que exista la oportunidad los gobiernos inventarán nuevos impuestos!).

Además de ser una clase software recién creada en el Modelo de Diseño, se trata de un concepto del dominio. Es normal y común descubrir conceptos del dominio relevantes y refinar el conocimiento de los requisitos durante el diseño o la programación —el desarrollo iterativo apoya esta clase de descubrimiento incremental—.

¿Debería reflejarse este descubrimiento en el Modelo del Dominio (o Glosario)? Si el Modelo del Dominio se utilizará en el futuro como fuente de inspiración para el trabajo de diseño posterior, o como ayuda visual para comunicar los conceptos claves del dominio, entonces podría ser útil añadirlo. La Figura 23.3 presenta un Modelo del Dominio actualizado.

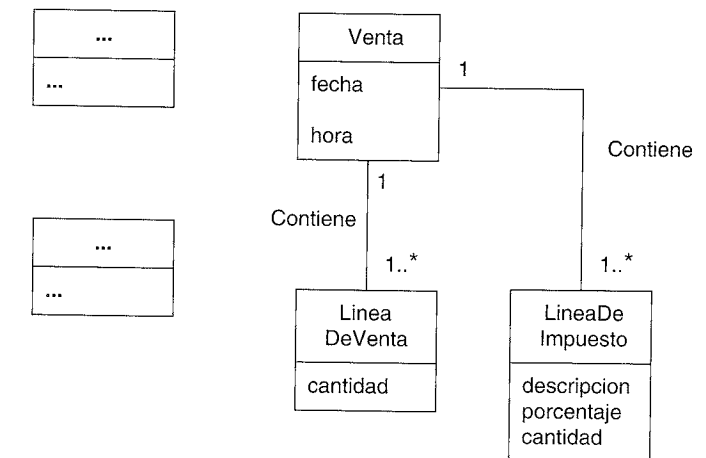


Figura 23.3. Modelo del Dominio parcial actualizado.

¿Mantenemos el Modelo del Dominio?

Puntualizando el comentario anterior sobre la actualización del Modelo del Dominio: obsérvese que la arquitectura del Modelo de Diseño normalmente se organizará en capas (lo que se discutirá con más detalle en un capítulo posterior). Una de estas capas de clases de diseño se denominará **capa del dominio**; esta capa contendrá las clases software cuyos nombres y estructuras se inspiraron en el vocabulario y los conceptos del dominio (*Venta*, *LineaDeImpuesto*, etcétera).

Sugerencia

Después de algunas iteraciones, el Modelo del Dominio —como primera fuente de inspiración para las clases del diseño en la capa del dominio del Modelo de Diseño— podría dejar de tener utilidad. Si la actualización del Modelo del Dominio para reflejar los cambios en el Modelo de Diseño deja de tener un valor práctico, piense en eliminarlo. Más bien, únicamente lleve a cabo un trabajo de ingeniería inversa (con una herramienta CASE de UML) para generar un diagrama de clases de la capa del dominio a partir de las clases de diseño del Modelo de Diseño. Aunque son clases software en lugar de clases conceptuales del dominio puras, reflejan el vocabulario del dominio más relevante que ha surgido en el diseño del software y, por tanto, un diagrama de clases UML de las clases del diseño en la capa del dominio del Modelo de Diseño puede ser una “aproximación” útil al verdadero Modelo del Dominio.

Por favor, no lo mal interprete: no sugerimos que se elimine definitivamente un Modelo del Dominio, sino más bien que considere si merece la pena mantenerlo, o es simplemente documentación para generar trabajo, y que conozca qué alternativas pueden ser útiles.

Patrones Relacionados

Un adaptador de recursos que oculta un sistema externo podría también ser considerado un objeto Fachada (otro patrón GoF que se expone en este capítulo), puesto que envuelve el acceso al subsistema o sistema con un único objeto (que es la esencia de la Fachada). Sin embargo, el nombre de adaptador está motivado especialmente cuando el objeto que envuelve facilita la adaptación a diversas interfaces externas.

23.3. Factoría (GoF)

El adaptador da lugar a un nuevo problema en el diseño: en la solución del patrón Adaptador anterior para servicios externos con diversas interfaces, ¿quién crea el adaptador? ¿Y cómo determinar qué clase de adaptador crear, como *AdaptadorMasterE-Impuestos* o *AdaptadorImpuestosPro*?

Si los crea algún objeto del dominio, las responsabilidades de los objetos del dominio exceden la lógica pura de la aplicación (como el cálculo del total de la venta) y entran en otras cuestiones relacionadas con la conexión con componentes software externos.

Este punto subraya otro principio de diseño fundamental (normalmente considerado un principio de diseño de arquitectura): diseño para mantener una **separación de intereses** (*separation of concerns*). Es decir, divida en módulos o separe intereses distintos en áreas diferentes, de manera que cada una tenga un propósito cohesivo. Por ejemplo, la capa del dominio de los objetos software destaca las responsabilidades centradas relativamente en la lógica de la aplicación, mientras que un grupo diferente de objetos es responsable de las cuestiones de conectividad con sistemas externos.

Por tanto, la elección de un objeto del dominio (como el *Registro*) para crear los adaptadores no está de acuerdo con el objetivo de separación de intereses, y disminuye su cohesión.

Notación UML: Observe el estilo del diagrama UML de la Figura 23.4 que incluye una nota que muestra el pseudocódigo detallado de *getAdaptadorCalculadorDeImpuestos*. Este estilo nos permite incluir detalles dinámicos sobre algoritmos en los diagramas de clases estáticos de manera que podría disminuir la necesidad de utilizar diagramas de interacción.

Una alternativa típica en este caso es aplicar el patrón **Factoría** (o **Factoría Concreta**), en el que se define un objeto Fabricación Pura “factoría” para crear los objetos.

Los objetos factoría tienen varias ventajas:

- Separan la responsabilidad de la creación compleja en objetos de apoyo (*helper*) cohesivos.
- Ocultan la lógica de creación potencialmente compleja.
- Permiten introducir estrategias para mejorar el rendimiento de la gestión de la memoria, como objetos caché o de reciclaje.

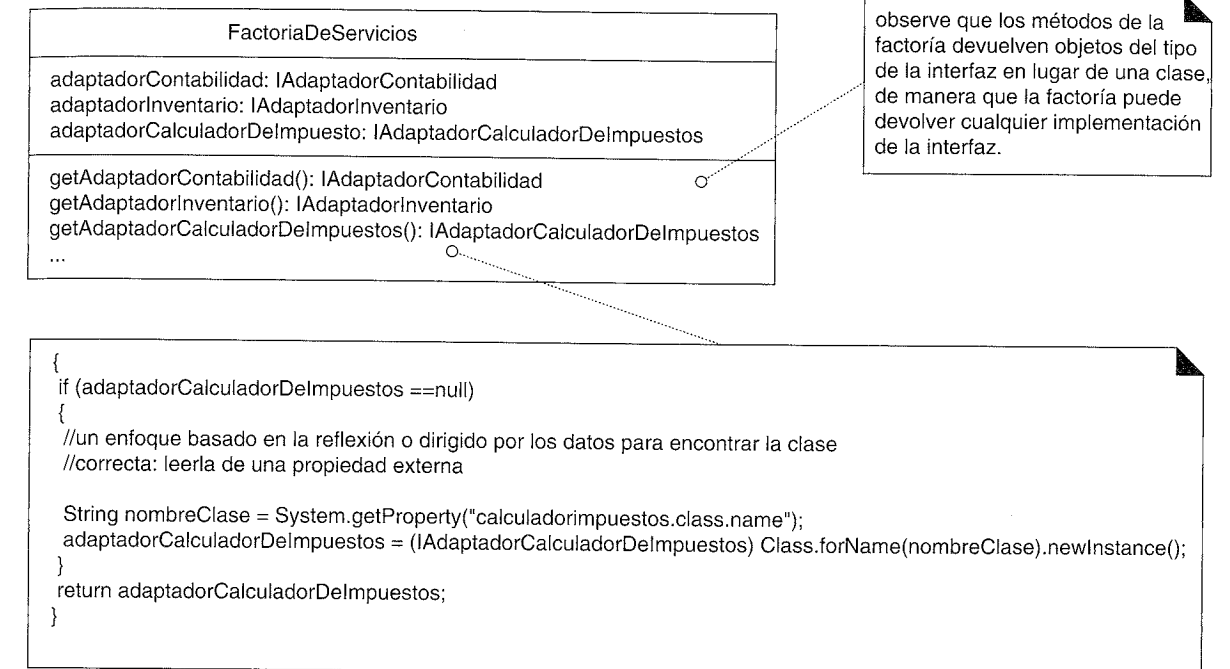
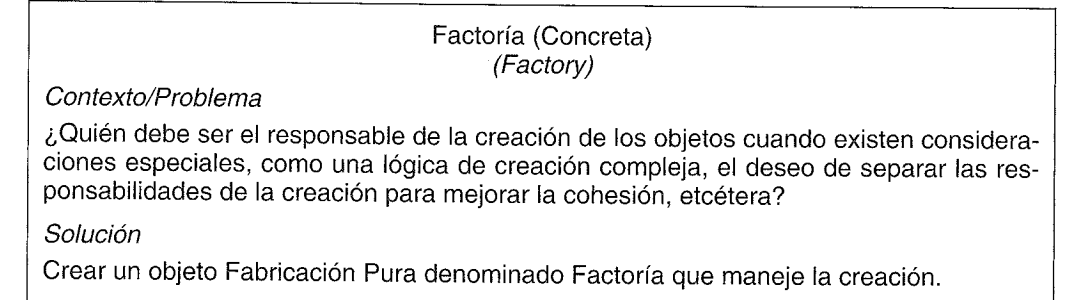


Figura 23.4. El patrón Factoría.



La solución de la Factoría se ilustra en la Figura 23.4.

Nótese que en la *FactoriaDeServicios*, la lógica para decidir qué clase se crea se resuelve leyendo el nombre de la clase desde una fuente externa (por ejemplo, por medio de una propiedad del sistema, si se utiliza Java) y después se carga la clase dinámicamente. Éste es un ejemplo de **diseño dirigido por los datos** parcial. El diseño consigue Variaciones Protegidas con respecto a los cambios en la clase de implementación del adaptador. Sin cambiar el código fuente en la clase factoría, podemos crear instancias de nuevas clases adaptador cambiando el valor de la propiedad y asegurando que la nueva clase es visible en la ruta de clases de Java para que pueda cargarse.

A menudo se accede a las factorías con el patrón Singleton⁴.

Patrones Relacionados

⁴ N. del T.: Al igual que para otros patrones, no hemos traducido el nombre de este patrón ya que a diferencia de otros (como Factoría o Adaptador) normalmente se utiliza el nombre original en inglés tanto en la expresión oral como escrita en castellano.

23.4. Singleton (GoF)

Con la *FactoriaDeServicios* surge un nuevo problema de diseño: ¿quién crea la propia factoría y cómo se accede?

En primer lugar, observe que sólo se necesita una instancia de la factoría en el proceso. Segundo, una rápida reflexión sugiere que los métodos de esta factoría podrían necesitar que se invoquen desde varios sitios del código, puesto que en diferentes lugares se necesita acceder a los adaptadores para solicitar los servicios externos. Por tanto, existe un problema de visibilidad: ¿cómo conseguir la visibilidad a esta única instancia de *FactoriaDeServicios*?

Una solución es pasar la instancia de *FactoriaDeServicios* como parámetro donde quiera que se descubra que se necesita que sea visible, o inicializar los objetos que necesitan que la factoría sea visible, con una referencia permanente. Esto es posible pero no es conveniente; una alternativa es el patrón **Singleton**.

Ocasionalmente, es conveniente mantener visibilidad global o un único punto de acceso a una única instancia de una clase, en lugar de cualquier otra forma de visibilidad. Esto se cumple para la instancia de *FactoriaDeServicios*.

Singleton	
<i>Contexto/Problema</i>	Se admite exactamente una instancia de una clase —es un “singleton”—. Los objetos necesitan un único punto de acceso global.
<i>Solución</i>	Defina un método estático de la clase que devuelva el singleton.

Por ejemplo, la Figura 23.5 muestra una implementación del patrón Singleton.

Por tanto, la idea clave es que la clase X defina un método estático *getInstancia* que él mismo proporciona una única instancia de X.

Con este enfoque, un desarrollador tiene visibilidad global a esta instancia única, por medio del método estático de la clase *getInstancia*, como vemos en este ejemplo:

```
public class Registro
{
    public void inicializar()
    {
        ... hace algún trabajo ...

        //acceso a la Factoría Singleton mediante la llamada a
        getInstancia adaptadorContabilidad =
            FactoriaDeServicios.getInstancia().getAdaptadorContabilidad();
    }
}
```

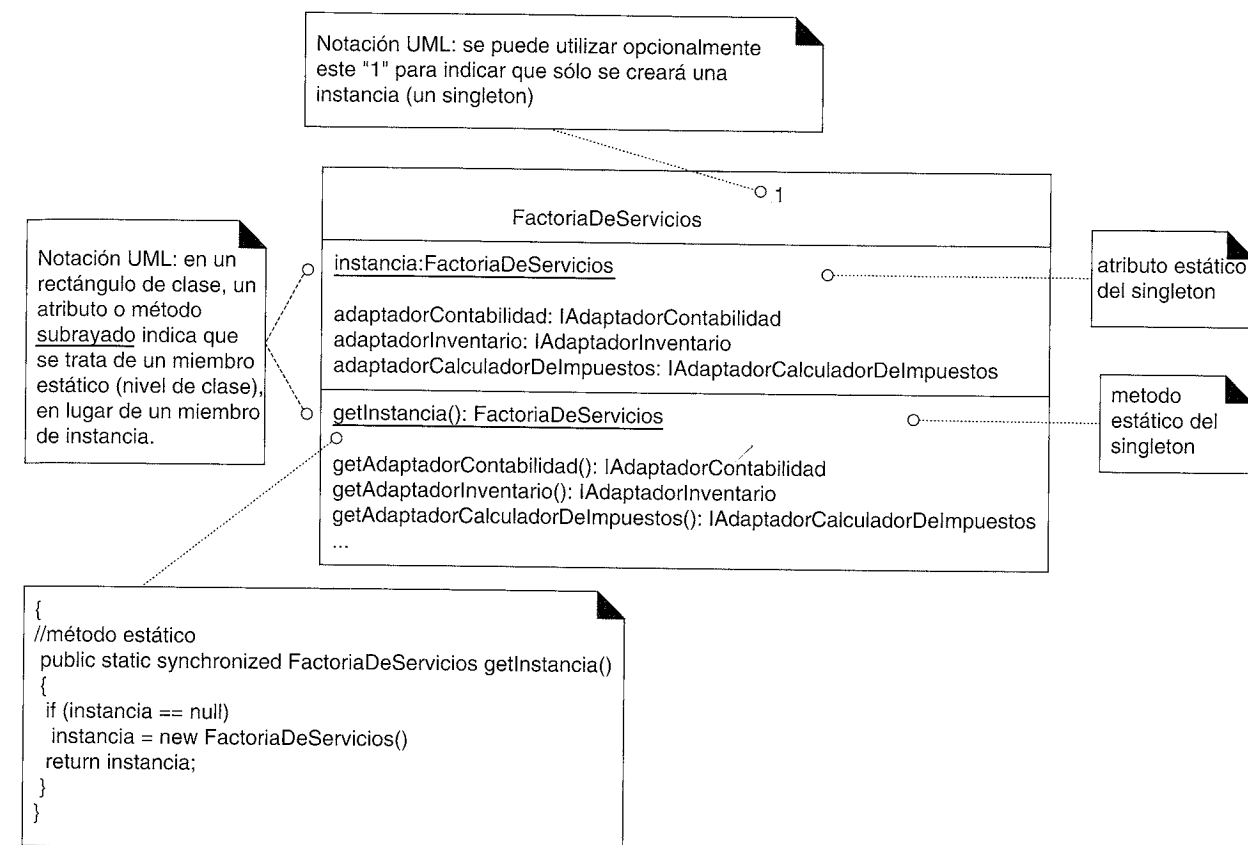


Figura 23.5. El patrón Singleton en la clase *FactoriaDeServicios*.

```
... hace algún trabajo ...
}

// otros métodos...
}
```

Puesto que la visibilidad de las clases públicas tiene un alcance global (en la mayoría de los lenguajes), en cualquier punto del código, en cualquier método de cualquier clase, uno puede escribir *ClaseSingleton.getInstancia()* para obtener la visibilidad de la instancia del singleton, y entonces enviarle un mensaje, como *ClaseSingleton.getInstancia().hacerFoo()*. Es difícil superar la emoción de ser capaces de *hacerFoo* globalmente.

Abreviatura UML para el acceso a un Singleton en los diagramas de interacción

La notación UML que implica —pero no muestra explícitamente— el mensaje *getInstancia* en un diagrama de interacción es añadir el estereotipo «singleton» a la instancia, como en la Figura 23.6. Este enfoque evita tener que mostrar explícitamente el mensaje (sin interés) *getInstancia* a la clase antes de enviar el mensaje a la instancia singleton.